

Desenvolvimento de Aplicações Web I

Sebenta da Unidade Curricular

João Pavão

Universidade de Trás-os-Montes e Alto Douro

Desenvolvimento de Aplicações Web I

Sebenta da Unidade Curricular

João Pavão

2026-08-22

Índice

Prefácio	1
Organização do documento	1
Fonte e atualização	1
Como usar este texto de apoio	2
Autor	2
1. Desenvolvimento de Software para o Ambiente Web	3
1.1. Perfis de Programador Web	3
1.1.1. Frontend	3
1.1.2. Backend	3
1.1.3. Full Stack	3
1.2. Ferramentas de Trabalho	4
1.3. HTML e CSS: os Blocos Básicos	4
1.4. Frameworks e Pré-processadores CSS	4
1.5. O Que um Frontend Iniciado Deve Saber	4
1.6. Tecnologias do Lado do Servidor	5
1.6.1. Linguagens	5
1.6.2. Frameworks de Servidor	5
1.6.3. Bases de Dados e ORMs	5
Referências	6
2. Arquiteturas Web e Infraestruturas	7
2.1. Arquiteturas Web	7
2.1.1. Arquitetura Cliente-Servidor	7
2.1.2. Model-View-Controller (MVC)	7
2.1.3. Arquitetura em Três Camadas	8
2.2. Infraestruturas	8
2.2.1. Servidores	8
2.2.2. Serviços Web (API)	9
2.2.3. Virtualização	9
2.2.4. O Ambiente de Trabalho desta UC	10
Referências	10

3. PHP I: Sintaxe, Variáveis e Arrays	11
3.1. Preparar o Ambiente de Desenvolvimento	11
3.1.1. Opção 1: Máquina Virtual	11
3.1.2. Opção 2: Contentores (Docker)	12
3.1.3. O Que Se Pretende Obter	12
3.2. A Linguagem PHP	12
3.3. Sintaxe Básica	13
3.4. Variáveis	13
3.5. Tipos de Dados	14
3.5.1. Conversão de Tipos	15
3.6. Arrays	15
3.6.1. Arrays Indexados	16
3.6.2. Arrays Associativos	16
3.6.3. Arrays Multidimensionais	17
3.6.4. Exemplo Completo: Média de um Aluno	17
Referências	18
4. PHP II: Condicionais, Loops e Supervariáveis	19
4.1. Estruturas Condicionais	19
4.1.1. Operadores de Comparação	20
4.1.2. Operadores Lógicos	21
4.2. Ciclos	21
4.2.1. <code>for</code>	21
4.2.2. <code>while</code>	21
4.2.3. <code>do...while</code>	22
4.2.4. <code>foreach</code>	22
4.3. Supervariáveis	22
Referências	23
5. PHP III: Funções, Datas e Includes	25
5.1. Funções	25
5.2. Datas	26
5.2.1. Obter a Data e Hora Atuais	26
5.2.2. Formatação Personalizada	26
5.2.3. Fusos Horários	27
5.3. Manipulação do Fluxo de Páginas: Includes	27
Referências	28
6. Modelos de Comunicação e Redes	29
6.1. Porque Organizar a Comunicação em Camadas?	29
6.2. Os Dois Modelos: OSI e TCP/IP	29
6.3. Serviços e Protocolos	30
6.4. O Percurso dos Dados: Encapsulamento	30
6.4.1. A Camada de Transporte: TCP	31
6.4.2. A Camada de Rede: IP	31
6.4.3. A Camada de Ligação: Ethernet e Endereços MAC	32
6.4.4. O Meio Físico	32
6.4.5. A Receção dos Dados	32
6.5. Encaminhamento (Routing)	32
6.6. Portas, Endereços e URLs	33
6.7. O Sistema de Nomes de Domínio (DNS)	33
6.8. Os Componentes da Web	33

Referências	34
7. O Protocolo HTTP e Gestão de Estado	35
7.1. O Protocolo HTTP	35
7.2. O Que Acontece Quando se Escreve um URL	35
7.3. A Mensagem de Pedido	35
7.3.1. O Método GET	36
7.3.2. O Método POST	36
7.3.3. GET vs. POST	37
7.3.4. Recursos Virtuais	37
7.4. Cabeçalhos HTTP	38
7.5. A Resposta do Servidor	38
7.6. HTTP é um Protocolo Stateless	38
7.7. Gestão de Estado: Cookies e Sessões	39
7.7.1. Cookies	39
7.7.2. Sessões	40
Referências	41
8. Bases de Dados e PHP	42
8.1. Bases de Dados Relacionais e o DBMS	42
8.2. Comandos SQL	42
8.3. A Arquitetura em Três Camadas e o Papel do PHP	43
8.4. Ligar o PHP ao MySQL	43
8.5. Criar Bases de Dados e Tabelas	44
8.6. Operações Fundamentais: CRUD em SQL	44
8.7. Prepared Statements	45
8.8. Erros e Boas Práticas	46
8.9. Uma Aplicação CRUD Completa	47
Referências	48
9. Segurança em Aplicações Web	49
9.1. Vulnerabilidades no Código Próprio	49
9.1.1. SQL Injection	49
9.1.2. XSS (Cross-Site Scripting)	50
9.1.3. CSRF (Cross-Site Request Forgery)	50
9.2. Vulnerabilidades em Dependências de Terceiros	51
9.3. Boas Práticas	51
Referências	51
10. Model-View-Controller (MVC)	53
10.1. Porquê Separar Responsabilidades?	53
10.2. Refazer o CRUD com MVC	53
10.3. Regras de Negócio no Model	54
10.4. O Controlador Frontal (Front Controller)	55
Referências	55
11. Frameworks de Desenvolvimento Web	57
11.1. O Que um Framework Automatiza	57
11.2. O Laravel	57
11.3. Outros Frameworks	58
Referências	59

Apêndices	60
A. Configuração do Ambiente de Desenvolvimento	60
A.1. Introdução	60
A.1.1. Sistemas Operativos dos Computadores dos Alunos	60
A.1.2. O phpMyAdmin	61
A.2. Máquina Virtual Fornecida pelo Professor	61
A.2.1. Instalar o VirtualBox	61
A.2.2. Importar o Ficheiro .ova	61
A.2.3. Primeiros Testes	62
A.3. Contentores Fornecidos pelo Professor	62
A.3.1. Instalar o Docker	62
A.3.2. Carregar e Arrancar os Contentores	63
A.3.3. Primeiros Testes	63
A.4. Construir o Ambiente a Partir do Zero	63
A.4.1. Opção 1: Máquina Virtual com VirtualBox	64
A.4.2. Opção 2: Contentores com Docker	66
A.5. Verificar a Instalação	68
Referências	68
B. Demos de Código das Aulas	70
B.1. Introdução	70
B.2. Demo: Variáveis e Arrays	70
B.3. Demo: Condicionais, Loops e Supervariáveis	71
B.4. Demo: Funções	73
B.5. Demo: Datas	74
B.6. Demo: Includes (Header e Footer)	74
Referências	75

Prefácio

Este documento é um documento de apoio ao estudo autónomo para a componente teórica da disciplina de **Desenvolvimento de Aplicações Web I** (DAW1), Licenciatura em Comunicação e Multimédia, cobrindo o desenvolvimento de aplicações web do lado do servidor com PHP, desde os fundamentos da comunicação cliente-servidor até à arquitetura e segurança de aplicações web modernas.

Na componente prática da UC, os alunos aplicam estes conceitos na construção de sítios web dinâmicos com PHP e bases de dados. Este documento cobre a componente teórica; a componente prática é acompanhada em separado, nas respetivas aulas.

Organização do documento

O texto está organizado em cinco blocos temáticos, que seguem o programa oficial da UC:

1. **Fundamentos e Panorama da Web** — o processo de desenvolvimento de uma aplicação web e os diferentes tipos de servidores e infraestruturas
2. **Programação em PHP** — sintaxe, controlo de fluxo, funções e organização de código
3. **Comunicação e Estado na Web** — modelos de rede, o protocolo HTTP e a gestão de estado com cookies e sessões
4. **Dados e Segurança** — bases de dados com PHP e segurança de aplicações web
5. **Arquitetura e Ferramentas** — o padrão Model-View-Controller e frameworks de desenvolvimento web

Fonte e atualização

Os conteúdos deste documento de apoio ao estudo baseiam-se no material próprio do docente, desenvolvido ao longo de anos anteriores de lecionação da UC (slides e demonstrações de código), reorganizado e atualizado para o ano letivo 2026/2027. A bibliografia de referência da UC está indicada em cada capítulo e compilada no ficheiro **referencias.bib**.

Como usar este texto de apoio

Cada capítulo corresponde a uma aula teórica da UC. A recomendação é que os alunos leiam um capítulo antes da aula respetiva, e que usem aeste texto como apoio ao estudo autónomo, para rever conceitos e esclarecer dúvidas. A leitura não substitui a presença nas aulas, nem a participação nas demonstrações práticas em sala de aula.

Autor

João Pavão — UTAD, docente responsável, componente teórica

1.

Desenvolvimento de Software para o Ambiente Web

1.1. Perfis de Programador Web

O desenvolvimento web organiza-se, tradicionalmente, em três perfis, que se distinguem pelo lado da aplicação em que trabalham.

1.1.1. Frontend

O programador de *frontend* trabalha do lado do cliente, isto é, no *browser*, e é responsável pela interface com que o utilizador interage diretamente. As ferramentas de base são o HTML, a CSS e o JavaScript, frequentemente com o apoio de *frameworks* como o React, o Vue.js ou o Angular. O React é hoje o mais usado, mas o Vue.js consolidou-se como segunda opção mais adotada globalmente, à frente do Angular em satisfação dos programadores («State of JavaScript 2025: Front-end Frameworks» 2025).

1.1.2. Backend

O programador de *backend* trabalha do lado do servidor: implementa a lógica de negócio, gere o acesso a bases de dados e expõe essa lógica ao exterior, tipicamente através da API do protocolo HTTP. Usa linguagens como o PHP, o Python ou o JavaScript (em ambiente Node.js), e sistemas de gestão de bases de dados como o MySQL, o MariaDB, o PostgreSQL ou o MongoDB. Ocupa-se ainda de tarefas de configuração de servidores e de implantação (*deployment*) da aplicação.

1.1.3. Full Stack

O programador *full stack* trabalha nos dois lados, frontend e backend, com as ferramentas acima descritas ou com *frameworks* que integram as duas componentes, como o Laravel (em PHP), o Django ou o FastAPI (em Python), ou o Next.js (em JavaScript/TypeScript).

1.2. Ferramentas de Trabalho

Independentemente do perfil, um programador web trabalha tipicamente com o seguinte conjunto de ferramentas:

- **Computador e sistema operativo:** macOS, Windows ou Linux;
- **Browser:** Chrome, Firefox, Safari ou Edge, para testar o resultado do trabalho;
- **Editor de texto ou IDE:** Visual Studio Code, Sublime Text ou uma IDE mais completa, como o WebStorm;
- **Terminal:** a linha de comandos do macOS ou Linux, ou o PowerShell ou Git Bash no Windows;
- **Software de design:** para protótipos e interfaces, sendo o Figma a ferramenta de referência do mercado (Mapsoft, s.d.).

1.3. HTML e CSS: os Blocos Básicos

O HTML e a CSS são os blocos de construção fundamentais de qualquer página web: o HTML estrutura o conteúdo e a CSS define a sua apresentação visual. Um programador de *frontend* deve dominar, no mínimo:

- As *tags* básicas e semânticas do HTML5;
- Os fundamentos da CSS: seletores, propriedades e o modelo de caixa;
- Alinhamentos e grelhas em CSS, com recurso a Flexbox e CSS Grid;
- *Media queries* e desenho responsivo, para que a página funcione em diferentes tamanhos de ecrã;
- Animações e transições simples.

1.4. Frameworks e Pré-processadores CSS

Escrever CSS pura para um projeto grande é possível, mas trabalhoso. Surgiram por isso *frameworks* CSS, bibliotecas de estilos e componentes prontos a usar. O Tailwind CSS é hoje o mais usado, à frente do Bootstrap («Top CSS Frameworks in 2025: What Devs Actually Use» 2025); o Foundation mantém-se como opção de nicho, sobretudo em projetos empresariais já existentes.

Para além de *frameworks* de componentes, existem ferramentas que estendem a própria linguagem CSS. O Sass é um pré-processador que acrescenta funcionalidades à CSS, como variáveis, aninhamento de seletores e *mixins*; o PostCSS transforma CSS através de plugins em JavaScript, com funcionalidades semelhantes. O aninhamento nativo de seletores, as variáveis nativas (**custom properties**) e funcionalidades como o `color-mix()` já fazem parte da própria linguagem CSS, com suporte alargado nos browsers modernos («Native CSS Nesting: Sass Syntax With No Build Step (And One Big Trap)» 2025), cobrindo grande parte do que motivava o uso de um pré-processador.

1.5. O Que um Frontend Iniciado Deve Saber

Resumindo o que se espera de um programador de *frontend* em início de carreira:

- Criar *layouts* web com HTML e CSS;

- Saber JavaScript e trabalhar com o DOM (*Document Object Model*);
- Criar pequenas aplicações de *frontend* interativas;
- Ser capaz de ligar essas aplicações a APIs externas;
- Trabalhar com o Git e usar controlo de versões;
- Instalar e gerir pequenos projetos reais, já em produção.

1.6. Tecnologias do Lado do Servidor

As tecnologias do lado do servidor organizam-se em três grandes grupos: linguagens e *frameworks*, bases de dados, e a programação através da API do protocolo HTTP.

1.6.1. Linguagens

Do lado do servidor, o panorama de linguagens inclui, entre outras, Node.js (JavaScript no servidor), Python, PHP, C#, Go, Ruby, Java, Kotlin, Rust, Scala e Swift. O PHP é uma das linguagens mais usadas do lado do servidor, sobretudo pela sua ligação histórica ao WordPress e a grande parte da Web publicada em sistemas de gestão de conteúdos.

1.6.2. Frameworks de Servidor

Cada linguagem de backend tem os seus próprios *frameworks* de desenvolvimento web. Em JavaScript, destacam-se o Express, o Koa, o Fastify e o Next.js; em Python, o Django, o Flask e o FastAPI; em PHP, o Laravel, o Symfony e o Slim. O Laravel é hoje o *framework* PHP mais usado, à frente do Symfony (JetBrains 2025).

1.6.3. Bases de Dados e ORMs

As bases de dados dividem-se, essencialmente, em dois grandes grupos:

- **Relacionais**, organizadas em tabelas com esquema fixo e relações entre elas, como o PostgreSQL, o MySQL ou o SQLite;
- **Não relacionais** (*NoSQL*), com esquemas mais flexíveis, como o MongoDB e o Firebase (Cloud Firestore), ambos orientados a documentos (Google, s.d.), ou o Redis (chave-valor, muito usado para *cache*).

Para facilitar o acesso a bases de dados a partir de código orientado a objetos, usam-se com frequência ferramentas de mapeamento objeto-relacional (ORM), como o Eloquent (o ORM nativo do Laravel), o Prisma, o Drizzle ou o TypeORM em JavaScript/TypeScript, o Mongoose para MongoDB, ou o SQLAlchemy em Python. O Prisma é o mais usado em número de projetos, mas o Drizzle tem crescido com destaque entre projetos que privilegiam um controlo mais direto sobre o SQL gerado (Bytebase 2026).

Referências

- Bytebase. 2026. «Drizzle ORM vs Prisma: Which TypeScript ORM Should You Use in 2026?». 2026. <https://www.bytebase.com/blog/drizzle-vs-prisma/>.
- DreamFactory. 2025. «45 SOAP to REST Conversion Statistics Every Enterprise Should Know». 2025. <https://www.dreamfactory.com/hub/soap-to-rest-conversion-statistics>.
- Google. s.d. «Cloud Firestore Data Model». <https://firebase.google.com/docs/firestore/data-model>.
- JetBrains. 2025. «The State of PHP 2025». 2025. <https://blog.jetbrains.com/phpstorm/2025/10/state-of-php-2025/>.
- Mapsoft. s.d. «Adobe XD End of Life: Status, Timeline & Alternatives». <https://mapsoft.com/adobe-xd-end-of-life/>.
- «Native CSS Nesting: Sass Syntax With No Build Step (And One Big Trap)». 2025. 2025. <https://www.w3tweaks.com/css/css-nesting-explained/>.
- OWASP Foundation. 2026. «OWASP Top 10:2025». 2026. <https://owasp.org/Top10/2025/>.
- Patchstack. 2025. «Patchstack’s 2025 WordPress Security Report: Mid-Year Vulnerability Breakdown». 2025. <https://patchstack.com/whitepaper/2025-mid-year-vulnerability-report/>.
- Reenskaug, Trygve. 2003. «MVC: Xerox PARC 1978-79». 2003. <https://citeseerx.ist.psu.edu/document?repid=rep1&type=pdf&doi=4ef90a7b9c1b1cd02acf273694e4059a70c7d198>.
- «State of JavaScript 2025: Front-end Frameworks». 2025. 2025. <https://2025.stateofjs.com/en-US/libraries/front-end-frameworks/>.
- «Top CSS Frameworks in 2025: What Devs Actually Use». 2025. 2025. <https://tailkits.com/blog/popular-css-frameworks/>.

2. Arquiteturas Web e Infraestruturas

2.1. Arquiteturas Web

2.1.1. Arquitetura Cliente-Servidor

O modelo cliente-servidor é o paradigma de arquitetura amplamente usado no desenvolvimento de aplicações web, no qual os papéis dos computadores ou dispositivos envolvidos se dividem em duas categorias:

- **Cliente:** dispositivo ou programa que faz pedidos a um servidor para obter serviços, recursos ou informação. É tipicamente o *browser*, em computadores, telemóveis ou tablets, e é responsável por interagir com o utilizador e enviar pedidos ao servidor.
- **Servidor:** programa que responde aos pedidos dos clientes, processando-os, realizando operações e enviando as respostas de volta.

Um só computador pode desempenhar os dois papéis ao mesmo tempo, correndo tanto o cliente como o servidor, e um servidor pode responder a muitos clientes em simultâneo. O que nunca muda é a direção do pedido: é sempre o cliente que inicia a comunicação, o servidor limita-se a responder. Um servidor não envia dados a um cliente por iniciativa própria, sem que antes exista um pedido.

Este modelo tem uma consequência direta na forma como uma aplicação web se organiza: o código que corre no cliente, HTML, CSS e JavaScript, é enviado pelo servidor mas depois executado inteiramente no computador do utilizador, fora do controlo do programador de *backend*; já o código que corre no servidor, neste caso PHP, nunca é visível ao cliente, que recebe apenas o resultado do seu processamento. Esta separação é a base de toda a arquitetura estudada nesta UC: o *frontend* corre no cliente, o *backend* corre no servidor, e os dois lados comunicam através do protocolo HTTP.

2.1.2. Model-View-Controller (MVC)

O MVC é um padrão de arquitetura de software, proposto originalmente por Trygve Reenskaug em 1978, durante o seu trabalho no Xerox PARC sobre a linguagem Smalltalk (Reenskaug 2003). Hoje é um dos padrões mais usados no desenvolvimento de aplicações web, e separa a aplicação em três componentes:

- **Model** (Modelo): representa os dados e a lógica de negócio da aplicação. Lida com o acesso à base de dados, o processamento de dados e as regras de negócio, sendo responsável por armazenar e recuperar informação.
- **View** (Visualização): responsável por apresentar os dados ao utilizador, cuidando da interface e da forma como a informação é mostrada. Pode ser uma página web, uma interface gráfica ou qualquer outro método de apresentação.
- **Controller** (Controlador): atua como intermediário entre o modelo e a visualização. Recebe os pedidos do utilizador, processa-os e decide o que fazer com base neles, atualizando o modelo conforme necessário e escolhendo a visualização apropriada para mostrar os resultados.

Numa aplicação PHP típica, o HTML e a CSS constituem a *View*, o PHP implementa o *Controller* (e, frequentemente, também parte do *Model*), e a base de dados, como o MySQL, é onde o *Model* persiste os dados.

A principal vantagem do MVC é a separação de responsabilidades: quem altera a aparência de uma página não precisa de mexer na lógica de negócio, e quem altera a forma como os dados são processados não precisa de mexer no HTML. Isto facilita a manutenção do código à medida que a aplicação cresce, permite que partes diferentes da aplicação sejam testadas de forma independente, e possibilita que várias pessoas trabalhem em simultâneo em componentes diferentes da mesma aplicação sem interferirem umas com as outras.

2.1.3. Arquitetura em Três Camadas

Intimamente relacionada com o MVC está a arquitetura em três camadas (*3-tier*), que separa uma aplicação em camada de apresentação (o que o utilizador vê e com que interage), camada de lógica de negócio (o processamento e as regras da aplicação) e camada de dados (o armazenamento persistente). Numa aplicação PHP, isto traduz-se, tipicamente, em HTML/CSS na camada de apresentação, PHP na camada de lógica, e MySQL na camada de dados.

A diferença entre o MVC e a arquitetura em três camadas é sobretudo de nível: o MVC é um padrão de organização do código dentro de uma aplicação, enquanto a arquitetura em três camadas descreve, de forma mais geral, como essas responsabilidades se distribuem, podendo inclusivamente correr em máquinas físicas diferentes, por exemplo um servidor de aplicação separado do servidor de base de dados. Na prática, para uma aplicação PHP como as desenvolvidas nesta UC, as duas arquiteturas descrevem, em grande medida, a mesma separação de responsabilidades vista de ângulos diferentes.

2.2. Infraestruturas

2.2.1. Servidores

Um servidor web é o programa responsável por escutar pedidos HTTP numa porta de rede e devolver respostas, tipicamente páginas web ou dados. É uma peça de *software* que corre continuamente numa máquina, à espera de novas ligações, capaz de lidar com muitos pedidos em simultâneo, vindos de clientes diferentes. Os servidores web mais usados incluem o **Apache** e o **NGinX**, ambos capazes de servir páginas PHP, seja através de um módulo embutido ou de um processo separado (como o **PHP-FPM**) que o servidor web invoca para executar o código PHP.

Para além dos servidores web, existem outros tipos de servidores especializados. No mundo Java, por exemplo, há servidores de aplicações como o **GlassFish**, mantido atualmente pela Eclipse Foundation no âmbito do Jakarta EE, e o **WildFly** (anteriormente conhecido como JBoss Application Server; hoje o nome do produto comercial suportado pela Red Hat é JBoss EAP). Do lado dos dados, o **MySQL Server** é o exemplo mais comum no contexto desta UC, correndo como um processo à parte do servidor web, e comunicando com o PHP através de uma ligação de rede local.

2.2.2. Serviços Web (API)

Uma API (*Application Programming Interface*) de um serviço web permite que diferentes sistemas comuniquem entre si através da rede, tipicamente sobre HTTP. Do ponto de vista da arquitetura MVC, uma API é, em muitos casos, um *Controller* que devolve dados estruturados, em vez de uma página HTML pronta a mostrar ao utilizador, para serem consumidos por outra aplicação. Os dois estilos mais conhecidos são:

- **REST** (*Representational State Transfer*): usa os métodos HTTP (GET, POST, PUT, DELETE) sobre recursos identificados por URLs, e troca dados normalmente em JSON. É hoje o estilo dominante, representando cerca de 75% da adoção entre programadores (DreamFactory 2025).
- **SOAP** (*Simple Object Access Protocol*): um protocolo mais rígido e formal, baseado em XML, com um contrato de serviço explícito. Está hoje limitado sobretudo a sistemas legados em indústrias reguladas, como a banca, a saúde ou sistemas empresariais (ERP), onde continua a processar volumes de transações elevados apesar de já não ser a escolha para novos serviços (DreamFactory 2025).

2.2.3. Virtualização

Para desenvolver e testar aplicações web de forma isolada do sistema operativo do computador local, e sem o risco de o resultado do trabalho depender de particularidades desse computador, usam-se ambientes virtualizados. Há duas abordagens principais:

Máquinas virtuais: cada máquina virtual inclui o seu próprio sistema operativo completo, para além da aplicação e das suas dependências. A vantagem é isolarem-se completamente umas das outras, e as suas configurações poderem ser partilhadas e reproduzidas com confiança, garantindo que todos os alunos de uma turma trabalham exatamente sobre a mesma configuração de servidor; a desvantagem é a duplicação de recursos, já que cada máquina virtual carrega o seu próprio sistema operativo, e tempos de arranque mais longos.

Contentores: em vez de replicarem todo o sistema operativo, os contentores partilham o núcleo do sistema operativo do anfitrião, encapsulando apenas a aplicação e as suas dependências. O **Docker** é a ferramenta de referência para este modelo. As vantagens são o menor impacto no sistema, o arranque mais rápido e a facilidade de partilha, compilação e distribuição; a desvantagem é um isolamento menos completo do que uma máquina virtual inteira, já que todos os contentores de uma máquina partilham o mesmo núcleo do sistema operativo.

Esta distinção não é apenas académica: as mesmas duas abordagens usadas para desenvolver localmente, numa máquina virtual ou num contentor, são também as mais comuns para pôr uma aplicação web em produção num servidor real, o que faz da escolha do ambiente de desenvolvimento uma decisão com impacto direto na forma como a aplicação será depois publicada.

2.2.4. O Ambiente de Trabalho desta UC

Nesta UC, o *backend* é desenvolvido dentro de uma máquina virtual (VirtualBox) com Ubuntu, contendo a *stack* XAMP (consoante o sistema operativo poderá ser WAMP para o Windows, MAMP para o MAC, XAMP para vários OS), Apache (o servidor), MySQL (a base de dados) e PHP (a linguagem de programação), as quatro iniciais que dão nome ao acrónimo. O editor de texto usado é o Visual Studio Code. Este ambiente isola o trabalho de desenvolvimento do sistema operativo do computador de cada aluno, seja macOS, Ubuntu ou Windows, garantindo que todos trabalham sobre a mesma configuração de servidor, base de dados e linguagem, independentemente do computador usado em casa.

Referências

- Bytebase. 2026. «Drizzle ORM vs Prisma: Which TypeScript ORM Should You Use in 2026?» 2026. <https://www.bytebase.com/blog/drizzle-vs-prisma/>.
- DreamFactory. 2025. «45 SOAP to REST Conversion Statistics Every Enterprise Should Know». 2025. <https://www.dreamfactory.com/hub/soap-to-rest-conversion-statistics>.
- Google. s.d. «Cloud Firestore Data Model». <https://firebase.google.com/docs/firestore/data-model>.
- JetBrains. 2025. «The State of PHP 2025». 2025. <https://blog.jetbrains.com/phpstorm/2025/10/state-of-php-2025/>.
- Mapsoft. s.d. «Adobe XD End of Life: Status, Timeline & Alternatives». <https://mapsoft.com/adobe-xd-end-of-life/>.
- «Native CSS Nesting: Sass Syntax With No Build Step (And One Big Trap)». 2025. 2025. <https://www.w3tweaks.com/css/css-nesting-explained/>.
- OWASP Foundation. 2026. «OWASP Top 10:2025». 2026. <https://owasp.org/Top10/2025/>.
- Patchstack. 2025. «Patchstack’s 2025 WordPress Security Report: Mid-Year Vulnerability Breakdown». 2025. <https://patchstack.com/whitepaper/2025-mid-year-vulnerability-report/>.
- Reenskaug, Trygve. 2003. «MVC: Xerox PARC 1978-79». 2003. <https://citeseerx.ist.psu.edu/document?repid=rep1&type=pdf&doi=4ef90a7b9c1b1cd02acf273694e4059a70c7d198>.
- «State of JavaScript 2025: Front-end Frameworks». 2025. 2025. <https://2025.stateofjs.com/en-US/libraries/front-end-frameworks/>.
- «Top CSS Frameworks in 2025: What Devs Actually Use». 2025. 2025. <https://tailkits.com/blog/popular-css-frameworks/>.

3. PHP I: Sintaxe, Variáveis e Arrays

3.1. Preparar o Ambiente de Desenvolvimento

Antes de escrever a primeira linha de código PHP, é necessário preparar um ambiente onde esse código possa ser executado: um servidor web capaz de interpretar PHP, ligado a um sistema de gestão de bases de dados. Como visto no Secção 2.2.3, há duas abordagens principais para isolar este ambiente do sistema operativo do computador de cada aluno, evitando que o resultado de um trabalho dependa de particularidades desse computador: máquinas virtuais e contentores. Esta secção descreve, de forma resumida, como cada uma destas abordagens se aplica especificamente ao desenvolvimento em PHP; os passos completos de instalação e configuração, com todos os comandos necessários, ficam reservados para o Secção A.1, já que são detalhes de configuração, não de programação, e podem variar consoante o sistema operativo do computador de cada aluno e a versão exata de cada ferramenta. Note-se ainda que o professor das aulas práticas pode implementar este ambiente com especificações mais concretas do que as aqui descritas; o objetivo desta secção é apenas garantir que o ambiente de programação em PHP fica claro em termos gerais, independentemente desses detalhes finais.

Na prática, não é necessário que cada aluno construa este ambiente do zero: o professor da componente prática disponibiliza normalmente uma imagem já preparada, seja de uma máquina virtual completa, seja de um ou mais contentores **Docker** (por exemplo, um contentor para o **Apache** e o **PHP**, e outro separado para o **MySQL**), prontos a importar e a usar. As duas opções descritas de seguida, com os passos completos no Secção A.1, destinam-se sobretudo a quem prefere montar o próprio ambiente, por exemplo num computador pessoal fora do contexto da aula, ou a quem simplesmente quer compreender, em detalhe, do que é feito o ambiente que lhe é disponibilizado.

3.1.1. Opção 1: Máquina Virtual

É a abordagem seguida por omissão nesta UC, já apresentada no Secção 2.2.4: uma máquina virtual **VirtualBox** com **Ubuntu**, onde se instalam o servidor **Apache**, o **MySQL** e o **PHP**, individualmente ou através de um pacote pré-configurado, consoante o sistema operativo do computador de cada aluno. Todo o desenvolvimento, desde a edição do código até ao teste no

browser, acontece dentro desta máquina virtual, que se comporta, para efeitos práticos, como um pequeno servidor próprio e isolado do resto do computador.

3.1.2. Opção 2: Contentores (Docker)

Uma alternativa é usar o **Docker**, referido no Secção 2.2.3, para correr o Apache, o PHP e o MySQL cada um no seu próprio contentor, em vez de os instalar dentro de uma máquina virtual completa. É uma opção mais leve, com arranque mais rápido, especialmente adequada a quem já tem alguma experiência com **Docker**, ou a um computador com menos recursos disponíveis para correr uma máquina virtual inteira.

3.1.3. O Que Se Pretende Obter

Seja qual for a opção escolhida, o resultado final é o mesmo, e é isso que importa reter para o resto deste capítulo:

- Um servidor **Apache** a correr, capaz de interpretar código PHP;
- Um servidor **MySQL** a correr, acessível a partir do código PHP;
- Um editor de código, tipicamente o **Visual Studio Code**, com acesso direto aos ficheiros do projeto;
- A possibilidade de abrir um ficheiro `.php` num *browser*, através de um endereço como `http://localhost`, e ver o resultado da sua execução.

Com este ambiente preparado, seja através da imagem fornecida pelo professor, seja seguindo os passos detalhados no Secção A.1, está reunido tudo o que é necessário para começar, finalmente, a escrever e a testar código PHP.

3.2. A Linguagem PHP

PHP (*Hypertext Preprocessor*) é uma linguagem de *script* que corre do lado do servidor. Ao contrário do HTML e do JavaScript, que o *browser* recebe e interpreta diretamente, o código PHP é executado no servidor, e o cliente recebe apenas o resultado desse processamento, tipicamente HTML. Isto significa que o código PHP nunca é visível a quem visita o sítio, mesmo que este veja o código-fonte da página no *browser*: tudo o que o cliente recebe já é o resultado da execução do PHP, nunca o código PHP em si.

Uma secção de código PHP começa sempre com `<?php` e pode ser fechada com `?>`, embora, em ficheiros que só contenham PHP, seja habitual omitir a marca de fecho, para evitar espaços em branco acidentais depois dela. O PHP pode ser inserido no meio de HTML, misturando as duas linguagens no mesmo ficheiro, entrando e saindo do modo PHP quantas vezes for necessário:

```
<!DOCTYPE html>
<html>
<body>
  <h1>Olá!</h1>
  <?php
    echo "<p>São agora " . date("H:i") . "</p>";
  ?>
  <p>Este texto já não é PHP, é HTML normal.</p>
```

```
<?php
    $visitas = 128;
    echo "<p>Este sítio já foi visitado $visitas vezes.</p>";
?>
</body>
</html>
```

3.3. Sintaxe Básica

Todos os comandos PHP terminam com ponto e vírgula (;); esquecê-lo é um dos erros mais comuns de quem começa a programar em PHP, e produz tipicamente um erro de sintaxe (*parse error*) que impede a página de carregar. Os comentários de uma linha fazem-se com `//`, e os de várias linhas com `/* ... */`:

```
<?php
// Este comentário de uma linha não aparece na página

/* Este comentário
   pode ocupar
   várias linhas */

echo "Olá, Mundo!";
```

Para mostrar um valor, usa-se o comando `echo`, que pode receber vários valores separados por vírgula:

```
<?php
echo "Olá, Mundo!";
echo "Nome: ", "João", ", Idade: ", 25;
```

3.4. Variáveis

Uma variável é um “contentor” para armazenar dados que o programa vai usar. Em PHP, o nome de uma variável tem de obedecer a um conjunto de regras precisas:

- Começa sempre pelo símbolo `$`;
- Depois do `$`, o nome tem de começar por uma letra ou por um *underscore* (`_`), nunca por um número;
- Só pode conter letras, números e *underscores*, sem espaços nem outros símbolos;
- É sensível a maiúsculas e minúsculas (*case sensitive*): `$idade` e `$Idade` são duas variáveis diferentes, um erro comum que pode gerar bastante confusão ao depurar um *script*.

```
<?php
$idade = 19;           // válido
$_total = 0;          // válido, começa por underscore
$nome_completo = "Ana Silva"; // válido

$1idade = 19;         // INVÁLIDO: não pode começar por número
```

3.5. Tipos de Dados

Ao contrário de linguagens como o Java ou o C#, o PHP não obriga a declarar o tipo de uma variável: o tipo é determinado automaticamente pelo valor atribuído, e pode mudar se a variável receber um valor de outro tipo mais tarde. A esta característica chama-se tipagem dinâmica. O PHP suporta os seguintes tipos de dados:

- **String:** texto, delimitado por aspas simples ou duplas;
- **Integer:** números inteiros, positivos ou negativos, sem casas decimais;
- **Float** (ou *double*): números com casas decimais;
- **Boolean:** os valores `true` ou `false`;
- **Array:** uma coleção de vários valores, tema detalhado na secção seguinte;
- **Object:** uma instância de uma classe, agrupando dados e comportamento;
- **NULL:** um tipo especial, com um único valor possível, `null`, que representa a ausência de valor;
- **Resource:** uma referência a um recurso externo ao PHP, como uma ligação aberta a uma base de dados ou a um ficheiro.

Os quatro primeiros, string, integer, float e boolean, são os tipos simples usados no dia a dia da programação; os restantes, array, object, NULL e resource, aparecem ao longo desta UC à medida que os programas se tornam mais complexos.

```
<?php
$idade = 19;           // integer
$preco_do_almoco = 10.5; // float
$texto = "Olá, Mundo!"; // string
$porta_aberta = true;  // boolean
$sem_valor = null;     // NULL
```

A função `gettype()` devolve o tipo atual de uma variável, o que é particularmente útil enquanto se aprende a linguagem, já que os erros de tipo em PHP nem sempre são óbvios:

```
<?php
echo gettype($idade); // integer
echo gettype($preco_do_almoco); // double
echo gettype($porta_aberta); // boolean
```

Um exemplo típico de confusão para quem começa: `echo` de um valor booleano mostra 1 para `true`, mas não mostra literalmente nada para `false`, o que pode parecer, à primeira vista, um erro do próprio programa, quando é apenas o comportamento normal do PHP.

Strings podem ser concatenadas com o operador `.`, e é possível misturar texto e variáveis diretamente numa mesma instrução `echo`, seja por concatenação, seja escrevendo a variável dentro de uma *string* entre aspas duplas:

```
<?php
$nome = "Ana";
$idade = 25;

echo "Nome: " . $nome . ", Idade: " . $idade . " anos"; // por concatenação
echo "Nome: $nome, Idade: $idade anos";                 // dentro de aspas duplas
```

Note-se que esta interpolação de variáveis só funciona com aspas duplas; dentro de aspas simples, `$nome` seria mostrado literalmente, sem ser substituído pelo seu valor.

3.5.1. Conversão de Tipos

Apesar de a tipagem em PHP ser dinâmica, é frequentemente necessário converter explicitamente um valor de um tipo para outro, por exemplo para garantir que um número recebido de um formulário HTML, que chega sempre como *string*, é tratado como número antes de ser usado num cálculo. Esta conversão faz-se colocando o tipo pretendido, entre parênteses, antes do valor:

```
<?php
$idade_texto = "25";           // string, como chegaria de um formulário
$idade = (int) $idade_texto;   // 25, agora um integer

$preco = "10.5";
$preco_num = (float) $preco;   // 10.5, agora um float

$numero = 0;
$estado = (bool) $numero;      // false: 0, "", "0" e null convertem-se a false

echo gettype($idade);         // integer
echo gettype($estado);        // boolean
```

O PHP também converte tipos automaticamente, sem ser preciso pedir, sempre que uma operação o exige: numa soma, uma *string* puramente numérica é convertida para número sem qualquer aviso:

```
<?php
$a = "10";
$b = "5";

echo $a + $b;    // 15 (inteiro), as strings são convertidas automaticamente
echo $a . $b;    // "105" (string), o operador . força a concatenação
```

Este comportamento é útil, mas também uma fonte comum de confusão para quem começa: o mesmo símbolo `+` soma números, mas o `.` concatena texto, e é fácil trocar um pelo outro sem que o PHP assinala qualquer erro, apenas produzindo um resultado inesperado.

3.6. Arrays

Um array é uma estrutura de dados que permite armazenar múltiplos valores relacionados numa única variável, acessíveis através de índices numéricos ou de chaves associativas, em vez de se criar uma variável separada para cada valor. O PHP disponibiliza três tipos de arrays.

3.6.1. Arrays Indexados

Num array indexado (ou numérico), cada valor é acessado através de um índice inteiro, que começa sempre em 0:

```
<?php
$frutas = array("Maçã", "Banana", "Laranja");

echo $frutas[0]; // Maçã
echo $frutas[1]; // Banana
echo $frutas[2]; // Laranja

$frutas[] = "Kiwi"; // acrescenta um novo elemento ao fim
echo count($frutas); // 4, o número de elementos do array
```

A partir do PHP 5.4, `array("Maçã", "Banana")` pode também ser escrito com a sintaxe abreviada `["Maçã", "Banana"]`, hoje a forma mais comum de o encontrar em código PHP recente.

Para além de `count()`, o PHP disponibiliza várias funções para trabalhar com o conteúdo de um array indexado, sem ter de percorrer os seus elementos manualmente:

```
<?php
$notas = [14, 18, 9, 16];

array_push($notas, 12); // acrescenta 12 ao fim, tal como $notas[] = 12
$ultima = array_pop($notas); // remove e devolve o último elemento (12)

sort($notas); // ordena por ordem crescente, reindexando o array
print_r($notas); // [9, 14, 16, 18]

if (in_array(16, $notas)) {
    echo "Encontrada uma nota de 16 valores";
}

$posicao = array_search(14, $notas); // devolve o índice onde está o 14
echo $posicao;
```

3.6.2. Arrays Associativos

Num array associativo, cada valor é acessado através de uma chave à escolha, tipicamente uma *string*, em vez de um índice numérico, o que torna o código mais legível quando cada valor tem um significado próprio:

```
<?php
$pessoa = array(
    "nome" => "João",
    "idade" => 25,
    "cidade" => "Lisboa"
);
```

```
echo $pessoa["nome"];    // João
echo $pessoa["idade"];   // 25

$pessoa["profissao"] = "Programador"; // acrescenta uma nova chave
```

3.6.3. Arrays Multidimensionais

Um array multidimensional é um array cujos valores são, eles próprios, outros arrays, o que permite representar dados mais estruturados, como categorias com listas de itens, ou uma lista de registos, cada um com vários campos:

```
<?php
$carros = array(
    "desportivos" => array("Ferrari", "Lamborghini", "Porsche"),
    "suv"          => array("BMW X5", "Audi Q7", "Mercedes GLE")
);

echo $carros["desportivos"][0]; // Ferrari
echo $carros["suv"][1];         // Audi Q7

// Um array de registos, cada um por sua vez um array associativo
$alunos = array(
    array("nome" => "Ana", "nota" => 16),
    array("nome" => "Rui", "nota" => 14),
    array("nome" => "Sara", "nota" => 18)
);

echo $alunos[0]["nome"]; // Ana
echo $alunos[2]["nota"]; // 18
```

Este último exemplo, um array de arrays associativos, é a forma mais comum de representar, em memória, um conjunto de registos vindos de uma base de dados, tema retomado com mais detalhe no Secção 8.1.

Para inspecionar rapidamente o conteúdo de um array durante o desenvolvimento, sem ter de aceder a cada elemento individualmente, usa-se a função `print_r()`, que mostra a sua estrutura completa, incluindo arrays aninhados, ou `var_dump()`, que mostra ainda o tipo e o comprimento de cada valor.

3.6.4. Exemplo Completo: Média de um Aluno

Um pequeno exemplo, juntando variáveis, tipos, conversão e arrays multidimensionais, mostra bem como estes conceitos se combinam na prática: calcular a média de um aluno a partir de um array de notas, guardado dentro de um array associativo com os seus dados:

```
<?php
$aluno = [
    "nome" => "Marta",
    "notas" => [15, 12, 18, 14]
```

```
];

$soma = array_sum($aluno["notas"]);           // 59
$numero_notas = count($aluno["notas"]);       // 4
$media = $soma / $numero_notas;               // 14.75

echo $aluno["nome"] . " tem média de " . $media;

if ($media >= 10) {
    echo ", está aprovado.";
} else {
    echo ", está reprovado.";
}
```

Note-se que `array_sum()` devolve um `float` sempre que a divisão não é exata, mesmo que todas as notas somadas sejam inteiros, o que é consistente com o comportamento de tipagem dinâmica visto na Secção 3.5: o PHP escolhe automaticamente o tipo mais adequado ao resultado de cada operação.

O código completo da demonstração feita na aula sobre variáveis e arrays, pronto a copiar, está disponível no Secção B.2.

Referências

- Bytebase. 2026. «Drizzle ORM vs Prisma: Which TypeScript ORM Should You Use in 2026?» 2026. <https://www.bytebase.com/blog/drizzle-vs-prisma/>.
- DreamFactory. 2025. «45 SOAP to REST Conversion Statistics Every Enterprise Should Know». 2025. <https://www.dreamfactory.com/hub/soap-to-rest-conversion-statistics>.
- Google. s.d. «Cloud Firestore Data Model». <https://firebase.google.com/docs/firestore/data-model>.
- JetBrains. 2025. «The State of PHP 2025». 2025. <https://blog.jetbrains.com/phpstorm/2025/10/state-of-php-2025/>.
- Mapsoft. s.d. «Adobe XD End of Life: Status, Timeline & Alternatives». <https://mapsoft.com/adobe-xd-end-of-life/>.
- «Native CSS Nesting: Sass Syntax With No Build Step (And One Big Trap)». 2025. 2025. <https://www.w3tweaks.com/css/css-nesting-explained/>.
- OWASP Foundation. 2026. «OWASP Top 10:2025». 2026. <https://owasp.org/Top10/2025/>.
- Patchstack. 2025. «Patchstack's 2025 WordPress Security Report: Mid-Year Vulnerability Breakdown». 2025. <https://patchstack.com/whitepaper/2025-mid-year-vulnerability-report/>.
- Reenskaug, Trygve. 2003. «MVC: Xerox PARC 1978-79». 2003. <https://citeseerx.ist.psu.edu/document?repid=rep1&type=pdf&doi=4ef90a7b9c1b1cd02acf273694e4059a70c7d198>.
- «State of JavaScript 2025: Front-end Frameworks». 2025. 2025. <https://2025.stateofjs.com/en-US/libraries/front-end-frameworks/>.
- «Top CSS Frameworks in 2025: What Devs Actually Use». 2025. 2025. <https://tailkits.com/blog/popular-css-frameworks/>.

4. PHP II: Condicionais, Loops e Supervariáveis

4.1. Estruturas Condicionais

Uma estrutura condicional é uma estrutura de controlo que permite tomar decisões com base em condições específicas: avalia se uma condição é verdadeira ou falsa, e executa o bloco de código correspondente, direcionando o fluxo do programa de acordo com o resultado dessa avaliação. Em PHP, a estrutura mais comum é o `if/else`:

```
<?php
$idade = 18;

if ($idade >= 18) {
    echo "É maior de idade.";
} else {
    echo "É menor de idade.";
}
```

Quando há mais do que duas alternativas, encadeia-se `elseif`:

```
<?php
$nota = 12;

if ($nota >= 18) {
    echo "Excelente";
} elseif ($nota >= 14) {
    echo "Bom";
} elseif ($nota >= 10) {
    echo "Suficiente";
} else {
    echo "Reprovado";
}
```

Quando se compara uma única variável com vários valores possíveis, é mais legível usar `switch`. Repare-se no uso do `break`: sem ele, a execução continuaria para o `case` seguinte mesmo

depois de encontrar uma correspondência, um comportamento chamado *fallthrough*, por vezes usado propositadamente, como nas duas primeiras linhas do exemplo, para agrupar vários casos sob a mesma resposta:

```
<?php
switch ($dia_semana) {
    case "Sábado":
    case "Domingo":
        echo "Fim de semana";
        break;
    case "Sexta":
        echo "Quase fim de semana";
        break;
    default:
        echo "Dia útil";
}
```

4.1.1. Operadores de Comparação

As condições assentam em operadores de comparação, que avaliam dois valores e devolvem sempre um booleano, `true` ou `false`:

Operador	Significado
<code>==</code>	igual (compara apenas o valor)
<code>===</code>	idêntico (compara o valor e o tipo)
<code>!=</code>	diferente (compara apenas o valor)
<code>!==</code>	não idêntico (compara o valor e o tipo)
<code><</code>	menor que
<code>></code>	maior que
<code><=</code>	menor ou igual a
<code>>=</code>	maior ou igual a

A distinção entre `==` e `===` é uma das fontes mais comuns de erros subtis em PHP, precisamente por o PHP converter automaticamente entre tipos ao usar `==`:

```
<?php
var_dump(0 == "abc");    // false na maioria das versões atuais do PHP
var_dump(0 == "0");      // true: "0" é convertido para o número 0
var_dump(0 === "0");     // false: tipos diferentes, integer vs string
var_dump("10" == "1e1"); // true: ambos convertidos para o mesmo número
var_dump("10" === "1e1");// false: são strings diferentes
```

Por esta razão, é boa prática usar `===` sempre que se quer garantir que dois valores são realmente iguais, tanto no valor como no tipo, evitando comparações que dependam de conversões automáticas nem sempre óbvias.

4.1.2. Operadores Lógicos

Condições podem ser compostas usando operadores lógicos, que combinam o resultado de duas ou mais condições:

- **and** (ou `&&`): verdadeiro apenas se ambas as condições forem verdadeiras;
- **or** (ou `||`): verdadeiro se pelo menos uma das condições for verdadeira;
- **xor**: verdadeiro se exatamente uma das duas condições for verdadeira, mas não ambas.

```
<?php
$idade = 20;
$tem_carta = true;

if ($idade >= 18 && $tem_carta) {
    echo "Pode conduzir.";
}
```

4.2. Ciclos

Um ciclo (ou *loop*) é uma estrutura de controlo que permite repetir a execução de um bloco de código várias vezes, até que uma condição especificada deixe de se verificar, ou até que um número predeterminado de repetições seja alcançado. O PHP disponibiliza quatro tipos de ciclos.

4.2.1. for

Usa-se quando se sabe, à partida, quantas repetições são necessárias, recebendo três parâmetros: o valor inicial, a condição de paragem, e o incremento aplicado a cada repetição:

```
<?php
for ($i = 1; $i <= 5; $i++) {
    echo $i . " ";
}
// 1 2 3 4 5
```

4.2.2. while

Recebe apenas uma condição, avaliada **antes** de cada repetição; se a condição for falsa logo à partida, o bloco de código nunca chega a executar:

```
<?php
$contador = 1;
while ($contador <= 5) {
    echo $contador . " ";
    $contador++;
}
```

4.2.3. do...while

Muito semelhante ao **while**, mas com a condição avaliada **depois** de cada repetição, o que garante que o bloco de código executa sempre pelo menos uma vez, mesmo que a condição seja falsa logo à partida:

```
<?php
$contador = 10;
do {
    echo $contador . " ";
    $contador++;
} while ($contador <= 5);
// 10 (executa uma vez, mesmo com a condição já falsa)
```

4.2.4. foreach

Usado especificamente para percorrer arrays, sem necessidade de gerir um índice manualmente:

```
<?php
$frutas = ["Maçã", "Banana", "Laranja"];
foreach ($frutas as $fruta) {
    echo $fruta . "<br>";
}
```

Com um array associativo, o **foreach** permite aceder simultaneamente à chave e ao valor de cada elemento:

```
<?php
$ pessoa = ["nome" => "João", "idade" => 25, "cidade" => "Lisboa"];
foreach ($pessoa as $chave => $valor) {
    echo "$chave: $valor<br>";
}
```

4.3. Supervariáveis

O PHP disponibiliza um conjunto de arrays predefinidos, conhecidos como supervariáveis (ou superglobais): variáveis automaticamente inicializadas pelo PHP, disponíveis em qualquer parte de um *script*, independentemente do contexto ou da função em que se esteja, sem necessidade de as declarar. São muito úteis para aceder a informação comum a toda a aplicação, como dados de formulários, *cookies*, sessões ou parâmetros da URL. As mais usadas são:

- **\$_GET**: contém os dados enviados através da URL, tipicamente por um formulário com `method="get"`, ou por parâmetros escritos diretamente no endereço, como `pagina.php?id=5`;
- **\$_POST**: contém os dados enviados no corpo de um pedido HTTP, tipicamente por um formulário com `method="post"`, sem ficarem visíveis na URL;

- `$_REQUEST`: combina os dados de `$_GET`, `$_POST` e dos *cookies* do pedido, útil quando não interessa saber por que método os dados chegaram;
- `$_SESSION`: guarda informação de sessão do utilizador, como o seu estado de autenticação, ao longo de páginas diferentes, tema aprofundado no Secção 7.7.2;
- `$_COOKIE`: contém os *cookies* enviados pelo cliente ao servidor, tema aprofundado no Secção 7.7.1;
- `$_SERVER`: contém informação sobre o servidor e sobre o pedido atual, como o endereço do cliente, o método HTTP usado, ou o caminho do próprio *script*.

```
<?php
// Um formulário HTML enviou um campo chamado "nome" por GET
if (isset($_GET["nome"])) {
    echo "Olá, " . $_GET["nome"] . "!";
} else {
    echo "Olá, visitante!";
}
```

O uso de `isset()` no exemplo anterior não é acidental: aceder diretamente a uma chave de `$_GET` ou `$_POST` que não exista gera um aviso em PHP, pelo que é boa prática verificar sempre se um determinado campo foi de facto enviado antes de o usar.

As supervariáveis são o ponto de ligação entre o HTML, do lado do cliente, e a lógica em PHP, do lado do servidor: um formulário preenchido pelo utilizador só produz efeito porque os seus campos chegam ao *script* PHP através de `$_GET` ou `$_POST`.

O código completo da demonstração feita na aula sobre condicionais, ciclos e supervariáveis, incluindo o formulário HTML usado para testar `$_GET` e `$_POST`, está disponível no Secção B.3.

Referências

- Bytebase. 2026. «Drizzle ORM vs Prisma: Which TypeScript ORM Should You Use in 2026?» 2026. <https://www.bytebase.com/blog/drizzle-vs-prisma/>.
- DreamFactory. 2025. «45 SOAP to REST Conversion Statistics Every Enterprise Should Know». 2025. <https://www.dreamfactory.com/hub/soap-to-rest-conversion-statistics>.
- Google. s.d. «Cloud Firestore Data Model». <https://firebase.google.com/docs/firestore/data-model>.
- JetBrains. 2025. «The State of PHP 2025». 2025. <https://blog.jetbrains.com/phpstorm/2025/10/state-of-php-2025/>.
- Mapsoft. s.d. «Adobe XD End of Life: Status, Timeline & Alternatives». <https://mapsoft.com/adobe-xd-end-of-life/>.
- «Native CSS Nesting: Sass Syntax With No Build Step (And One Big Trap)». 2025. 2025. <https://www.w3tweaks.com/css/css-nesting-explained/>.
- OWASP Foundation. 2026. «OWASP Top 10:2025». 2026. <https://owasp.org/Top10/2025/>.
- Patchstack. 2025. «Patchstack's 2025 WordPress Security Report: Mid-Year Vulnerability Breakdown». 2025. <https://patchstack.com/whitepaper/2025-mid-year-vulnerability-report/>.
- Reenskaug, Trygve. 2003. «MVC: Xerox PARC 1978-79». 2003. <https://citeseerx.ist.psu.edu/document?repid=rep1&type=pdf&doi=4ef90a7b9c1b1cd02acf273694e4059a70c7d198>.
- «State of JavaScript 2025: Front-end Frameworks». 2025. 2025. <https://2025.stateofjs.com/en-US/libraries/front-end-frameworks/>.

Referências

«Top CSS Frameworks in 2025: What Devs Actually Use». 2025. 2025. <https://tailkits.com/blog/popular-css-frameworks/>.

5. PHP III: Funções, Datas e Includes

5.1. Funções

Uma função é um conjunto de instruções nomeado, a que se dá um nome, que pode ser chamado num programa para realizar uma tarefa específica. As funções ajudam a organizar e a reutilizar código, melhorando a legibilidade e a manutenção do programa: em vez de repetir o mesmo bloco de instruções em vários pontos de um *script*, escreve-se uma vez, dentro de uma função, e chama-se essa função sempre que necessário. Uma função pode aceitar parâmetros (dados de entrada) e devolver um valor (dado de saída), ambos opcionais.

```
<?php
function saudacao($nome) {
    return "Olá, " . $nome . "!";
}

echo saudacao("Ana");    // Olá, Ana!
echo saudacao("Rui");    // Olá, Rui!
```

Uma função pode receber vários parâmetros, e pode definir-lhes um valor por omissão, usado quando esse parâmetro não é indicado na chamada:

```
<?php
function somar($a, $b = 10) {
    return $a + $b;
}

echo somar(5);           // 15, usa o valor por omissão de $b
echo somar(5, 20);       // 25, o valor indicado sobrepõe-se ao valor por omissão
```

Uma função pode ainda não devolver nada, limitando-se a produzir um efeito, como mostrar diretamente algo no ecrã:

```
<?php
function mostrarLinha($texto) {
    echo "<p>" . $texto . "</p>";
}

mostrarLinha("Primeira linha");
mostrarLinha("Segunda linha");
```

Uma variável definida dentro de uma função é local a essa função, e não é visível fora dela, mesmo que uma variável com o mesmo nome exista também fora da função; a única forma de uma função comunicar um resultado para o resto do *script* é através do valor que devolve com `return`:

```
<?php
function calcularIVA($preco) {
    $taxa = 0.23; // $taxa só existe dentro desta função
    return $preco * (1 + $taxa);
}

echo calcularIVA(10); // 12.3
// echo $taxa; // erro: $taxa não está definida aqui fora
```

O código completo da demonstração feita na aula sobre funções está disponível no Secção [B.4](#).

5.2. Datas

O PHP oferece funções dedicadas à manipulação de datas e horas, particularmente relevantes numa aplicação web, onde é comum registar quando um contacto foi criado, calcular prazos, ou apresentar uma data num formato legível. Há três operações comuns sobre datas em PHP.

5.2.1. Obter a Data e Hora Atuais

A função `date()` formata a data e hora atuais do servidor de acordo com um padrão indicado como parâmetro, usando letras com significados predefinidos, como `d` para o dia, `m` para o mês, `Y` para o ano com quatro dígitos, `H` para a hora, `i` para os minutos e `s` para os segundos:

```
<?php
echo date("d/m/Y"); // 24/08/2026
echo date("d/m/Y H:i:s"); // 24/08/2026 15:42:07
echo date("l, d \d\e F"); // segunda-feira, 24 de Agosto (com localização)
```

5.2.2. Formatação Personalizada

Como se vê nos exemplos anteriores, o mesmo instante pode ser apresentado de formas muito diferentes, apenas mudando o formato pedido a `date()`, o que é útil para adaptar a apresentação de uma data ao contexto: um formato compacto para uma tabela, ou um formato por extenso para o cabeçalho de um artigo.

5.2.3. Fusos Horários

Por omissão, o PHP usa o fuso horário configurado no servidor, que pode não corresponder ao fuso horário dos utilizadores da aplicação. Pode definir-se explicitamente o fuso horário a usar com `date_default_timezone_set()`, logo no início do *script*:

```
<?php
date_default_timezone_set("Europe/Lisbon");
echo date("H:i:s"); // hora local em Portugal, independentemente do fuso do servidor
```

Para operações mais complexas sobre datas, como calcular a diferença entre duas datas ou adicionar um intervalo de tempo a uma data, o PHP disponibiliza a classe `DateTime`, orientada a objetos:

```
<?php
$data = new DateTime("2026-08-24");
$data->modify("+7 days");
echo $data->format("d/m/Y"); // 31/08/2026

$inicio = new DateTime("2026-08-01");
$fim = new DateTime("2026-08-24");
$diferenca = $inicio->diff($fim);
echo $diferenca->days . " dias"; // 23 dias
```

O código completo da demonstração feita na aula sobre datas, incluindo o cálculo de anos bissextos, está disponível no Secção [B.5](#).

5.3. Manipulação do Fluxo de Páginas: Includes

À medida que um sítio cresce, torna-se comum que várias páginas partilhem os mesmos elementos, como o cabeçalho ou o rodapé. Em vez de repetir esse código em cada ficheiro, o PHP permite incluir o conteúdo de um ficheiro dentro de outro, através de duas instruções muito semelhantes: `include` e `require`.

include: inclui o conteúdo de um ficheiro dentro do ficheiro atual. Se o ficheiro indicado não for encontrado, o PHP emite um aviso, mas o *script* continua a ser executado. É útil quando se quer incluir ficheiros opcionais, não críticos para o funcionamento do *script*.

require: funciona de forma semelhante ao `include`, mas é usado quando o ficheiro incluído é essencial para o funcionamento do *script*. Se o ficheiro indicado não for encontrado, o PHP emite um erro fatal e interrompe imediatamente a execução. É útil quando se quer garantir que um ficheiro fundamental está disponível antes de continuar a executar o resto do *script*.

```
<?php
require "header.php";
?>

<p>Conteúdo específico desta página.</p>

<?php
```

```
require "footer.php";  
?>
```

Como o cabeçalho e o rodapé costumam ser essenciais à estrutura visual de uma página, sem os quais o resultado ficaria incompleto ou quebrado, é comum preferir-se `require` a `include` para os incluir. Existem ainda as variantes `include_once` e `require_once`, que garantem que um mesmo ficheiro só é incluído uma vez, mesmo que a instrução apareça, direta ou indiretamente, mais do que uma vez no fluxo de execução, evitando erros como a redefinição de uma mesma função.

Este padrão, ficheiros `header.php` e `footer.php` incluídos em cada página do sítio, é o primeiro passo prático para pensar numa aplicação web como um conjunto de componentes reutilizáveis, em vez de páginas HTML independentes e desligadas umas das outras, uma ideia retomada com mais profundidade no Secção 2.1.2 e no Secção 10.1.

O código completo dos três ficheiros desta demonstração está disponível no Secção B.6.

Referências

- Bytebase. 2026. «Drizzle ORM vs Prisma: Which TypeScript ORM Should You Use in 2026?». 2026. <https://www.bytebase.com/blog/drizzle-vs-prisma/>.
- DreamFactory. 2025. «45 SOAP to REST Conversion Statistics Every Enterprise Should Know». 2025. <https://www.dreamfactory.com/hub/soap-to-rest-conversion-statistics>.
- Google. s.d. «Cloud Firestore Data Model». <https://firebase.google.com/docs/firestore/data-model>.
- JetBrains. 2025. «The State of PHP 2025». 2025. <https://blog.jetbrains.com/phpstorm/2025/10/state-of-php-2025/>.
- Mapsoft. s.d. «Adobe XD End of Life: Status, Timeline & Alternatives». <https://mapsoft.com/adobe-xd-end-of-life/>.
- «Native CSS Nesting: Sass Syntax With No Build Step (And One Big Trap)». 2025. 2025. <https://www.w3tweaks.com/css/css-nesting-explained/>.
- OWASP Foundation. 2026. «OWASP Top 10:2025». 2026. <https://owasp.org/Top10/2025/>.
- Patchstack. 2025. «Patchstack's 2025 WordPress Security Report: Mid-Year Vulnerability Breakdown». 2025. <https://patchstack.com/whitepaper/2025-mid-year-vulnerability-report/>.
- Reenskaug, Trygve. 2003. «MVC: Xerox PARC 1978-79». 2003. <https://citeseerx.ist.psu.edu/document?repid=rep1&type=pdf&doi=4ef90a7b9c1b1cd02acf273694e4059a70c7d198>.
- «State of JavaScript 2025: Front-end Frameworks». 2025. 2025. <https://2025.stateofjs.com/en-US/libraries/front-end-frameworks/>.
- «Top CSS Frameworks in 2025: What Devs Actually Use». 2025. 2025. <https://tailkits.com/blog/popular-css-frameworks/>.

6. Modelos de Comunicação e Redes

6.1. Porque Organizar a Comunicação em Camadas?

Antes de um pedido HTTP, tema do próximo capítulo, poder sair do *browser* e chegar a um servidor, é preciso que uma pilha inteira de protocolos funcione corretamente por baixo dele: encontrar o endereço do servidor, estabelecer uma ligação fiável, converter os dados em sinais elétricos ou impulsos de rádio, e fazer tudo isto sem que quem escreve uma aplicação web precise de pensar em nenhum destes detalhes. Esta separação é conseguida organizando a comunicação em rede em camadas.

Numa arquitetura em camadas, cada camada é um módulo que depende das camadas abaixo de si, e é completamente independente das camadas acima de si. Cada camada presta um **serviço** à camada imediatamente superior, sem que esta precise de saber como esse serviço é implementado por baixo. Esta organização pode ser usada em modo estrito, em que cada camada só comunica com a camada imediatamente abaixo de si, ou num modo mais flexível, em que uma camada pode aceder a qualquer uma das camadas inferiores.

A vantagem principal desta organização é a independência entre camadas: o protocolo HTTP, na camada de aplicação, não precisa de saber se os dados vão viajar por um cabo de cobre, por fibra ótica ou por Wi-Fi, isso é um problema das camadas inferiores; e a camada responsável pelo meio físico não precisa de perceber nada de HTTP, apenas de transportar bits de um lado para o outro. Isto permite que cada camada evolua, seja substituída ou seja otimizada de forma independente das restantes, desde que o serviço que presta às camadas acima de si se mantenha o mesmo.

6.2. Os Dois Modelos: OSI e TCP/IP

Historicamente, surgiram dois modelos de referência para organizar a comunicação em camadas, com origens diferentes:

- **OSI** (*Open Systems Interconnection*): desenvolvido pela ISO (*International Organization for Standardization*) como um modelo formal, com sete camadas: física, ligação de dados, rede, transporte, sessão, apresentação e aplicação. Por ter sido definido e aprovado formalmente por um organismo internacional de normalização antes de estar amplamente

implementado, o OSI é conhecido como o padrão *de jure* (por lei, ou seja, formalmente instituído).

- **TCP/IP** (também chamado modelo DoD, por ter as suas origens no Departamento de Defesa dos EUA): um modelo mais simples, com apenas quatro camadas, aplicação, transporte, rede e acesso à rede, que nasceu diretamente da implementação prática da ARPANET, a precursora da Internet. Por se ter tornado o modelo efetivamente usado, através da sua adoção generalizada, antes de qualquer normalização formal internacional, o TCP/IP é conhecido como o padrão *de facto* (de facto, ou seja, imposto pela prática e pelo uso corrente).

Na prática, a Internet de hoje funciona sobre o modelo TCP/IP, não sobre o OSI: o OSI continua a ser usado, sobretudo, como ferramenta pedagógica, por dividir de forma mais fina responsabilidades que, no TCP/IP, aparecem concentradas numa única camada. Para os objetivos desta UC, interessam sobretudo três camadas do modelo TCP/IP:

- **Aplicação:** onde correm os protocolos que uma aplicação usa diretamente, como o HTTP, o correspondente aproximado às três camadas superiores do OSI (aplicação, apresentação e sessão);
- **Transporte:** responsável por entregar os dados de forma fiável entre dois programas, através do protocolo TCP;
- **Rede:** responsável por encaminhar os dados entre redes diferentes, até ao seu destino, através do protocolo IP.

6.3. Serviços e Protocolos

Dois conceitos são centrais para perceber como as camadas comunicam entre si: serviços e protocolos.

Um **serviço** representa um conjunto de funções oferecidas a um utilizador por um fornecedor. Esse serviço é disponibilizado através de **Pontos de Acesso ao Serviço** (*SAPs*, *Service Access Points*): da perspetiva de quem usa o serviço, todas as suas características ficam completamente definidas pela interface no SAP, o que significa que quem fornece o serviço pode ser tratado como uma entidade completamente abstrata, sem que se precise de saber como esse serviço é implementado por dentro. É exatamente este princípio que permite a um programador de aplicações web usar o TCP sem precisar de perceber como este garante, internamente, a entrega fiável dos dados.

Um **protocolo**, por sua vez, é uma família de normas para a troca de informação: um conjunto de regras que definem as ligações físicas, os formatos de dados, os modelos de transmissão, e os meios usados para assegurar a correção de erros e falhas na transmissão de dados. Enquanto o serviço descreve *o quê* uma camada oferece à camada acima de si, o protocolo descreve *como* duas entidades da mesma camada, uma do lado do cliente e outra do lado do servidor, comunicam entre si para implementar esse serviço.

6.4. O Percurso dos Dados: Encapsulamento

Quando uma aplicação envia dados, estes atravessam cada camada do modelo TCP/IP, e cada camada acrescenta a sua própria informação de controlo, num processo chamado encapsulamento. Vale a pena seguir este percurso passo a passo, das camadas superiores até ao meio físico.

6.4.1. A Camada de Transporte: TCP

O *payload* (nome técnico dado aos dados, neste caso, por exemplo, uma mensagem HTTP) é passado à camada de transporte pela aplicação, através de um *socket*. É-lhe adicionado um cabeçalho TCP, cuja informação mais importante são as **portas de origem e de destino**. As portas variam entre 0 e 65535, e as chamadas *well-known ports*, entre 0 e 1023, estão reservadas para serviços conhecidos, como:

- **80**: HTTP;
- **443**: HTTPS;
- **21**: FTP;
- **23**: Telnet;
- **25**: SMTP (correio eletrónico).

Quando o conjunto de dados a enviar é muito extenso, o TCP divide-o em segmentos, cada um com um comprimento máximo de 64kB, e atribui a cada segmento um número sequencial. O TCP do lado recetor pode assim ordenar os segmentos pela ordem correta, verificar se todos chegaram, e montar de novo o conjunto de dados original, mesmo que os segmentos tenham chegado fora de ordem pela rede.

Sempre que o TCP recetor recebe um segmento, confirma ao TCP emissor que o recebeu, indicando o seu número sequencial; esta confirmação pode ser feita segmento a segmento, ou em grupos de vários segmentos de cada vez. Se um determinado segmento não for confirmado dentro de um prazo, o TCP emissor volta a enviá-lo, o que garante a entrega fiável dos dados, mesmo que a rede por baixo perca ocasionalmente algum pacote.

O TCP usa ainda um mecanismo de **janela** (*window*) para controlar o fluxo e a congestão: uma janela é um conjunto de segmentos que pode ser enviado de uma só vez, por exemplo 100 segmentos, sem esperar por uma confirmação individual de cada um antes de enviar o seguinte. Se o recetor demorar muito tempo a confirmar a receção, ou se a rede começar a apresentar sinais de congestionamento, o tamanho da janela é reduzido automaticamente, diminuindo a velocidade de envio; quando a rede volta a estar livre, o tamanho da janela aumenta de novo. Desta forma, o TCP ajusta-se sozinho às condições da rede em cada momento, sem que a aplicação que o usa precise de fazer nada.

6.4.2. A Camada de Rede: IP

A seguir, os dados passam para a camada de rede (a camada Internet), à qual é adicionado um cabeçalho IP. A informação mais importante deste cabeçalho são os **endereços IP de origem e de destino**. Um endereço IPv4 tem a forma (0-255) . (0-255) . (0-255) . (0-255), como em 192.168.24.36; um mesmo computador pode ter vários endereços IP, um por cada interface de rede que possuir.

O cabeçalho IP inclui ainda vários outros campos com funções específicas, entre os quais se destacam:

- **Número de saltos** (*TTL, Time To Live*): o número máximo de saltos (passagens por um *router*) que um pacote pode dar antes de ser descartado, um mecanismo de segurança para evitar que um pacote mal encaminhado circule indefinidamente pela rede;
- **Fragmentação**: cada rede tem um tamanho máximo de pacote que consegue transportar (o *MTU, Maximum Transmission Unit*); se um pacote IP ultrapassar esse tamanho, é fragmentado em partes menores, um processo realizado pelos nós intermédios da rede;

- **Protocolo:** indica se o conteúdo do pacote, na sua secção de dados, é um segmento TCP ou um datagrama UDP, informando a camada de transporte, do lado recetor, como deve interpretar o que se segue.

6.4.3. A Camada de Ligação: Ethernet e Endereços MAC

Depois da camada de rede vem a camada de ligação de dados, cuja tarefa é entregar um pacote a uma máquina dentro da mesma rede local. Esta camada adiciona os **endereços MAC** de origem e de destino, formando uma trama (*frame*), tipicamente segundo a norma IEEE 802.3 (Ethernet) ou o equivalente para redes sem fios.

6.4.4. O Meio Físico

Por fim, na camada física, os bits são convertidos em sinais elétricos, impulsos de luz (fibra ótica) ou ondas de rádio (Wi-Fi), e transmitidos pelo canal físico correspondente. É apenas neste último passo que os dados deixam, de facto, de ser informação abstrata e passam a ser um sinal físico a viajar por um cabo ou pelo ar.

6.4.5. A Receção dos Dados

Do lado recetor, este processo acontece exatamente ao contrário, camada a camada: a camada física deteta um sinal com o endereço Ethernet correspondente à interface de rede local, e passa a informação à camada de ligação; esta verifica se o endereço MAC de destino é o seu e, em caso afirmativo, passa a informação à camada de rede; esta verifica se o endereço IP de destino é o seu e, em caso afirmativo, passa a informação à camada de transporte; e esta, por fim, verifica se existe alguma aplicação registada na porta de destino indicada e, em caso afirmativo, entrega-lhe finalmente os dados. Cada camada remove o cabeçalho que a camada correspondente lhe tinha acrescentado do lado do emissor, até restarem apenas os dados originais da aplicação.

6.5. Encaminhamento (Routing)

Quando um computador recebe um pacote cujo endereço IP de destino não é o seu, uma de duas coisas pode acontecer: ou assume que houve um engano e descarta o pacote, ou, se tiver capacidade para tal, reencaminha-o para outra rede. A este segundo tipo de equipamento chama-se **router**.

Um exemplo típico: alguém, em casa, escreve uma pesquisa no **Google**. O pacote formado tem como IP de destino o IP dos servidores do **Google**. O *router* de casa recebe esse pacote, reconhece que o IP de destino não é o seu, e reencaminha-o para a rede pública; de router em router, de salto em salto, o pacote acaba por chegar ao computador de destino, com o IP pretendido. Para decidir para onde reencaminhar cada pacote, cada router mantém uma **tabela de encaminhamento** (*routing table*), com regras que associam destinos possíveis à interface de rede por onde o pacote deve seguir.

Vale a pena notar a diferença entre um *router* e um *switch*: um *switch* interliga vários equipamentos dentro da mesma rede local, encaminhando tramas com base no endereço MAC; um *router* interliga redes diferentes entre si, encaminhando pacotes com base no endereço IP.

É por isso que um pedido a um servidor fora da rede local, como o do Google no exemplo anterior, passa sempre por, pelo menos, um router.

6.6. Portas, Endereços e URLs

Um endereço como `http://www.utad.pt` esconde, na verdade, duas informações distintas: o endereço do servidor e a porta a usar. Quando nenhuma porta é indicada explicitamente, assume-se a porta bem conhecida do protocolo em causa, a porta 80 no caso do HTTP, pelo que `http://www.utad.pt` e `http://www.utad.pt:80` são, na prática, equivalentes.

Como os humanos memorizam mais facilmente texto do que números, o endereço escrito num URL tem sempre a forma de um nome de domínio, como `www.utad.pt`, nunca de um número IP diretamente. Mas a rede, ao nível da camada IP, só entende números. É o DNS que faz a ponte entre estas duas realidades.

6.7. O Sistema de Nomes de Domínio (DNS)

O DNS (*Domain Name System*) é o serviço responsável por traduzir nomes de domínio, como `www.utad.pt`, no endereço IP correspondente. O processo é simples do ponto de vista de quem o usa: o *browser* pergunta a um servidor de nomes qual é o IP correspondente a um dado domínio, e o sistema de nomes responde com o número IP, que o *browser* passa então a usar no cabeçalho IP de todos os pacotes seguintes para esse destino.

O DNS não é, contudo, um único servidor, mas um sistema hierárquico e distribuído de servidores, organizado por domínios e subdomínios: um servidor pode não saber diretamente o IP de `www.utad.pt`, mas sabe a que outro servidor perguntar, mais próximo desse domínio na hierarquia, até que a resposta final seja obtida. Só depois de resolvido o nome de domínio no seu endereço IP é que o pedido HTTP pode, de facto, ser enviado ao servidor de destino, o que faz da resolução DNS um passo obrigatório, e por vezes o mais lento, sempre que se acede pela primeira vez a um determinado sítio.

6.8. Os Componentes da Web

Tudo o que foi visto neste capítulo, camadas, encapsulamento, portas, endereços e DNS, é a infraestrutura sobre a qual assenta a própria Web. A Web, como sistema, assenta em três componentes fundamentais:

- Uma linguagem de apresentação: o HTML;
- Um esquema de endereçamento: o URI/URL;
- Um protocolo de comunicação entre cliente e servidor: o HTTP.

Um URL (*Uniform Resource Locator*) identifica, de forma única, um recurso na Web, combinando o protocolo a usar (`http`, `https`), o nome do servidor (que o DNS traduz para um IP), opcionalmente a porta, e o caminho para o recurso pretendido dentro desse servidor. É precisamente o protocolo HTTP, construído sobre toda a infraestrutura vista neste capítulo, que é o tema do próximo.

Referências

- Bytebase. 2026. «Drizzle ORM vs Prisma: Which TypeScript ORM Should You Use in 2026?». 2026. <https://www.bytebase.com/blog/drizzle-vs-prisma/>.
- DreamFactory. 2025. «45 SOAP to REST Conversion Statistics Every Enterprise Should Know». 2025. <https://www.dreamfactory.com/hub/soap-to-rest-conversion-statistics>.
- Google. s.d. «Cloud Firestore Data Model». <https://firebase.google.com/docs/firestore/data-model>.
- JetBrains. 2025. «The State of PHP 2025». 2025. <https://blog.jetbrains.com/phpstorm/2025/10/state-of-php-2025/>.
- Mapsoft. s.d. «Adobe XD End of Life: Status, Timeline & Alternatives». <https://mapsoft.com/adobe-xd-end-of-life/>.
- «Native CSS Nesting: Sass Syntax With No Build Step (And One Big Trap)». 2025. 2025. <https://www.w3tweaks.com/css/css-nesting-explained/>.
- OWASP Foundation. 2026. «OWASP Top 10:2025». 2026. <https://owasp.org/Top10/2025/>.
- Patchstack. 2025. «Patchstack’s 2025 WordPress Security Report: Mid-Year Vulnerability Breakdown». 2025. <https://patchstack.com/whitepaper/2025-mid-year-vulnerability-report/>.
- Reenskaug, Trygve. 2003. «MVC: Xerox PARC 1978-79». 2003. <https://citeseerx.ist.psu.edu/document?repid=rep1&type=pdf&doi=4ef90a7b9c1b1cd02acf273694e4059a70c7d198>.
- «State of JavaScript 2025: Front-end Frameworks». 2025. 2025. <https://2025.stateofjs.com/en-US/libraries/front-end-frameworks/>.
- «Top CSS Frameworks in 2025: What Devs Actually Use». 2025. 2025. <https://tailkits.com/blog/popular-css-frameworks/>.

7. O Protocolo HTTP e Gestão de Estado

7.1. O Protocolo HTTP

HTTP (*Hypertext Transfer Protocol*) é o protocolo da camada de aplicação que permite que duas aplicações comuniquem entre si na Web. Na maior parte das vezes, uma dessas aplicações é um servidor web (como o Apache, o Tomcat ou o GlassFish) e a outra é um *browser* (como o Chrome, o Firefox ou o Edge). Funciona segundo um modelo de pedido-resposta: o cliente envia um pedido HTTP ao servidor, e o servidor devolve uma resposta HTTP.

7.2. O Que Acontece Quando se Escreve um URL

Vale a pena seguir, passo a passo, o que acontece entre o momento em que um utilizador escreve um endereço no *browser* e o momento em que a página aparece no ecrã, juntando tudo o que foi visto no capítulo anterior:

1. O *browser* extrai o nome do servidor (*hostname*) do URL, incluindo o número de porta, se indicado, e consulta o DNS para obter o seu endereço IP;
2. Abre um *socket* TCP, especificando o IP de destino e a porta a usar, por omissão a porta 80 para uma página web comum;
3. Escreve uma mensagem de pedido HTTP e envia-a através desse *socket* TCP; se não for indicado explicitamente o nome de uma página no URL, o servidor assume, por omissão, `index.html`;
4. A camada TCP aceita a mensagem e garante a sua entrega fiável ao servidor, tal como descrito no Secção 6.4.1;
5. O servidor responde, enviando uma mensagem de resposta que percorre o caminho inverso, através da mesma pilha de protocolos, até chegar de volta ao *browser*, que a interpreta e apresenta a página ao utilizador.

7.3. A Mensagem de Pedido

Uma mensagem de pedido HTTP completa tem a seguinte estrutura geral:

[método] [URL] [versão]
[cabeçalhos]

[corpo]

O método indica a intenção do cliente. Os mais usados no desenvolvimento web são o GET e o POST.

7.3.1. O Método GET

O GET pede a obtenção de um recurso, sem que essa operação, por si só, altere nada no servidor. É o método usado sempre que se clica num link numa página:

```
<a href="http://utad.pt/index.html">UTAD</a>
```

Este link, ao ser clicado, resulta na seguinte mensagem GET:

```
GET / HTTP/1.1  
Host: utad.pt
```

Por não alterar nada no servidor, a mensagem GET é considerada uma mensagem **inócua**. Esta é uma convenção do protocolo, não uma garantia técnica: um servidor mal concebido pode, tecnicamente, alterar dados como resposta a um GET, mas fazê-lo viola o comportamento esperado por qualquer cliente HTTP, e é considerado má prática.

O GET pode também ser usado num contexto de formulário HTML, desde que o objetivo não seja alterar informação no servidor, apenas obter um resultado, como numa pesquisa:

```
<form action="/search" method="GET">  
  <label for="term">Procurar:</label>  
  <input id="term" name="term" type="text" />  
  <input type="submit" value="Procurar!" />  
</form>
```

Ao submeter este formulário com o termo “love”, o *browser* gera a seguinte mensagem:

```
GET /search?term=love HTTP/1.1  
Host: searchengine.com
```

Note-se como os dados do formulário aparecem diretamente na URL, depois do ?, no formato chave=valor.

7.3.2. O Método POST

O POST envia dados para o servidor, tipicamente para criar ou alterar um recurso. O seu uso típico é também num formulário HTML:

```
<form action="/account/create" method="POST">
  <label for="firstName">Nome próprio</label>
  <input id="firstName" name="firstName" type="text" />
  <label for="lastName">Apelido</label>
  <input id="lastName" name="lastName" type="text" />
</form>
<input type="submit" value="Registar!"/>
```

Que resulta na seguinte mensagem:

```
POST /account/create HTTP/1.1
Host: servername.com
```

```
firstName=Scott&lastName=Allen
```

Ao contrário do **GET**, os dados vão no corpo da mensagem, não na URL. Um cliente web envia tipicamente um pedido **POST** sempre que tem dados para enviar que vão produzir um efeito no servidor: por exemplo, ao clicar num botão “Adicionar ao carrinho” numa loja *online*, é enviado um **POST** com a informação do artigo escolhido. Ao contrário do **GET**, o **POST não é uma mensagem inócua**: tipicamente atualiza uma conta, regista um pedido ou altera de alguma outra forma o estado do servidor.

7.3.3. GET vs. POST

A escolha entre **GET** e **POST** num formulário tem consequências práticas concretas:

- **Desvantagem do GET**: os valores dos campos do formulário ficam visíveis na URL do *browser*. Por esta razão, **o GET nunca deve ser usado se os dados do formulário forem sensíveis**, como uma palavra-passe, e não devam ser partilhados ou ficar registados no histórico do *browser*;
- **Vantagem do GET**: precisamente por os parâmetros ficarem na URL, esta pode ser marcada como favorita ou partilhada, permitindo voltar aos mesmos resultados sempre que necessário, algo que o **POST** não permite diretamente.

7.3.4. Recursos Virtuais

Nem todos os recursos acessíveis através da Web existem, como tal, num ficheiro no servidor. Muitos são criados, ou montados “na hora”, de acordo com o pedido específico do cliente, depois de o servidor recuperar os dados necessários de uma base de dados, tema do Secção 8.1. Um pedido a `pagina.php?id=5`, por exemplo, não corresponde a um ficheiro chamado literalmente assim; corresponde à execução do *script* `pagina.php`, que, com base no parâmetro `id`, gera dinamicamente o conteúdo a devolver.

7.4. Cabeçalhos HTTP

Tanto o pedido como a resposta incluem cabeçalhos, pares de chave e valor com informação adicional sobre a mensagem. O único cabeçalho obrigatório num pedido é o `Host`, que indica a que servidor o pedido se destina, essencial quando um mesmo endereço IP aloja vários domínios diferentes. Outro cabeçalho comum é o `Accept`, que indica ao servidor que tipos de conteúdo (formatos `MIME`) o *browser* prefere receber na resposta, por ordem de preferência, indicada por um valor de qualidade `q` entre 0 e 1 (o valor por omissão é 1, a preferência máxima).

7.5. A Resposta do Servidor

A resposta do servidor segue uma estrutura semelhante à do pedido, mas começa por uma linha de estado, em vez de um método e um URL:

```
[versão] [código de estado] [frase de razão]
[cabeçalhos]
```

```
[corpo]
```

O código de estado é um número que resume o resultado do pedido, agrupado em cinco categorias:

- **1xx**: informativo, usado raramente diretamente por quem programa aplicações web;
- **2xx**: sucesso, como 200 `OK`;
- **3xx**: redireccionamento, como 301 `Moved Permanently`, que indica que o recurso pedido passou a estar disponível noutra endereço;
- **4xx**: erro do lado do cliente, como 404 `Not Found`, quando o recurso pedido não existe, ou 403 `Forbidden`, quando o acesso é negado;
- **5xx**: erro do lado do servidor, como 500 `Internal Server Error`, que indica uma falha na aplicação do lado do servidor, tipicamente um erro no próprio código PHP.

7.6. HTTP é um Protocolo Stateless

O servidor HTTP não guarda, por si só, qualquer informação sobre o estado de ligações anteriores: depois de responder a um pedido, o servidor “esquece” essa ligação, uma propriedade chamada *stateless* (sem estado). Isto tem uma consequência prática relevante: uma única página web normalmente exige vários recursos, o próprio HTML, a folha de estilos CSS, o *script* JavaScript, eventuais imagens, e cada um destes recursos exige um pedido HTTP independente, cada um com o custo de estabelecer a sua própria ligação.

Para atenuar este custo, os *browsers* recorrem a diferentes estratégias de gestão de ligações:

- **Ligações paralelas**: ao detetar vários recursos numa página, o *browser* abre várias ligações em simultâneo para os carregar mais depressa, uma para o HTML, outra para o CSS, outra para o JavaScript, e assim por diante. O número de ligações paralelas é limitado, porque cada ligação adicional aumenta a carga sobre o servidor, o que, a partir de certo ponto, pode tornar o carregamento da página mais lento em vez de mais rápido;

- **Ligações persistentes:** em vez de fechar o *socket* logo a seguir a cada resposta, o servidor mantém-no aberto para vários pedidos sucessivos. O Apache, por omissão, mantém uma ligação persistente aberta durante 5 segundos de inatividade antes de a fechar. Este período tem também de ser limitado por razões de segurança, para evitar ataques de negação de serviço (*DoS*, *Denial of Service*), em que um atacante mantém propositadamente muitas ligações abertas em simultâneo, impedindo o servidor de atender pedidos legítimos;
- **Ligações em rajada** (*back-to-back*): desde o HTTP/1.1, um cliente pode enviar vários pedidos seguidos, sobre a mesma ligação, sem esperar pela resposta a cada um antes de enviar o seguinte, reduzindo ainda mais o tempo total de carregamento de uma página com muitos recursos.

7.7. Gestão de Estado: Cookies e Sessões

Apesar de o HTTP ser, por natureza, *stateless*, o comportamento mais comum na Web é precisamente o oposto: um sítio de comércio eletrónico precisa de saber que artigos um utilizador colocou no seu carrinho, e um sistema de autenticação precisa de reconhecer um utilizador já autenticado, sem lhe pedir a palavra-passe em cada página que visita.

Antigamente, este seguimento de utilizador podia ser feito com base no endereço IP do cliente: se um pedido vinha de um determinado IP, o servidor assumia tratar-se sempre da mesma sessão. Com a popularização do NAT (*Network Address Translation*), esta abordagem deixou de funcionar: numa rede doméstica típica, vários dispositivos, cada um com o seu IP privado, partilham um único IP público perante o resto da Internet, pelo que pedidos de utilizadores completamente diferentes podem chegar ao servidor com o mesmo endereço IP de origem.

Surgiram então duas soluções, complementares, para manter o estado de uma sessão entre pedidos sucessivos: cookies e sessões.

7.7.1. Cookies

Os *cookies* estão definidos na norma RFC 6265, *HTTP State Management Mechanism*. Funcionam da seguinte forma: na primeira vez que um utilizador acede a um sítio, o servidor pode enviar-lhe um *cookie* através de um cabeçalho da resposta HTTP. A partir desse momento, o *browser* passa a incluir esse *cookie* em todos os pedidos seguintes a esse mesmo servidor, permitindo-lhe reconhecer o cliente.

```
HTTP/1.1 200 OK
```

```
Set-Cookie: GUID=00b38b9a6f949a8df5476999c876789e; domain=.msn.com; path=/
```

A partir daqui, o *browser* passa a enviar este cabeçalho em todos os pedidos seguintes a esse servidor:

```
GET http://www.msn.com HTTP/1.1
```

```
Host: msn.com
```

```
Cookie: GUID=00b38b9a6f949a8df5476999c876789e;
```

Um servidor pode definir vários *cookies* ao mesmo tempo, cada um com o seu próprio significado, por exemplo um identificador de sessão e a língua preferida do utilizador:

HTTP/1.1 200 OK

Set-Cookie: SID=31d4d96e407aad42; Path=/; Secure; HttpOnly

Set-Cookie: lang=pt-PT; Path=/; Domain=example.com

Um *cookie* definido pelo servidor pode incluir vários atributos, para além do par nome-valor:

- **Expires** ou **Max-Age**: definem quando o *cookie* deve expirar (uma data concreta, ou um número de segundos, respetivamente); se ambos forem definidos, prevalece o **Max-Age**. Sem nenhum dos dois, o *cookie* expira quando o *browser* é fechado;
- **Domain** e **Path**: restringem a que *host* e a que pasta do servidor o *cookie* é enviado;
- **Secure**: o *browser* só envia este *cookie* por um canal seguro, HTTPS;
- **HttpOnly**: o *cookie* só é trocado entre *browser* e servidor, sem ficar acessível a código JavaScript do lado do cliente, uma proteção importante contra ataques de XSS, tema do Secção 9.1.

Para apagar um *cookie*, o servidor reenvia-o com uma data de expiração já no passado; a partir desse momento, o *browser* deixa de o incluir nos pedidos seguintes. Note-se ainda que os *cookies*, por si só, não constituem autenticação: apenas permitem reconhecer que um pedido vem do mesmo *browser* que recebeu o *cookie* anteriormente, e têm uma limitação de 4kB de informação cada. Em PHP, um *cookie* cria-se com a função `setcookie()`:

```
<?php
setcookie(
    "utilizador",    // nome
    "joao",          // valor
    time() + 3600,   // expira em 1 hora
    "/",             // path
    "",              // domain
    true,            // secure
    true             // httponly
);
```

7.7.2. Sessões

As sessões vão mais longe do que os *cookies* simples: em vez de guardar os dados propriamente ditos no cliente, visíveis e alteráveis por quem tiver acesso ao *browser*, o servidor guarda os dados localmente, associados a um identificador único, e envia apenas esse identificador ao cliente, tipicamente através de um *cookie*. Em PHP, uma sessão inicia-se com `session_start()`, chamado no início de todas as páginas que a usem, e os seus dados ficam disponíveis através da supervariável `$_SESSION`:

```
<?php
session_start();
$_SESSION["utilizador"] = "joao";
```

```
<?php
// Noutra página, também com session_start() no início:
session_start();
echo "Bem-vindo, " . $_SESSION["utilizador"];
```

Cookies e sessões são, assim, a solução de engenharia que permite simular continuidade numa aplicação web construída sobre um protocolo que, por natureza, não guarda memória de nada entre um pedido e o seguinte.

Referências

- Bytebase. 2026. «Drizzle ORM vs Prisma: Which TypeScript ORM Should You Use in 2026?» 2026. <https://www.bytebase.com/blog/drizzle-vs-prisma/>.
- DreamFactory. 2025. «45 SOAP to REST Conversion Statistics Every Enterprise Should Know». 2025. <https://www.dreamfactory.com/hub/soap-to-rest-conversion-statistics>.
- Google. s.d. «Cloud Firestore Data Model». <https://firebase.google.com/docs/firestore/data-model>.
- JetBrains. 2025. «The State of PHP 2025». 2025. <https://blog.jetbrains.com/phpstorm/2025/10/state-of-php-2025/>.
- Mapsoft. s.d. «Adobe XD End of Life: Status, Timeline & Alternatives». <https://mapsoft.com/adobe-xd-end-of-life/>.
- «Native CSS Nesting: Sass Syntax With No Build Step (And One Big Trap)». 2025. 2025. <https://www.w3tweaks.com/css/css-nesting-explained/>.
- OWASP Foundation. 2026. «OWASP Top 10:2025». 2026. <https://owasp.org/Top10/2025/>.
- Patchstack. 2025. «Patchstack’s 2025 WordPress Security Report: Mid-Year Vulnerability Breakdown». 2025. <https://patchstack.com/whitepaper/2025-mid-year-vulnerability-report/>.
- Reenskaug, Trygve. 2003. «MVC: Xerox PARC 1978-79». 2003. <https://citeseerx.ist.psu.edu/document?repid=rep1&type=pdf&doi=4ef90a7b9c1b1cd02acf273694e4059a70c7d198>.
- «State of JavaScript 2025: Front-end Frameworks». 2025. 2025. <https://2025.stateofjs.com/en-US/libraries/front-end-frameworks/>.
- «Top CSS Frameworks in 2025: What Devs Actually Use». 2025. 2025. <https://tailkits.com/blog/popular-css-frameworks/>.

8. Bases de Dados e PHP

8.1. Bases de Dados Relacionais e o DBMS

Um DBMS (*Database Management System*, sistema de gestão de bases de dados) é o *software* que cria, organiza e gere bases de dados: controla o acesso, as consultas e as atualizações, garante a integridade, a segurança e o desempenho dos dados, e permite que aplicações, como as escritas em PHP, leiam e escrevam informação de forma persistente. Exemplos comuns de DBMS incluem o MySQL, o MariaDB, o PostgreSQL, o Oracle, o SQL Server e o SQLite.

Numa base de dados relacional, como o MySQL, os dados organizam-se em tabelas, cada uma com colunas de tipo fixo, e linhas que representam registos individuais. É esta persistência que falta às variáveis PHP vistas até agora: uma variável existe apenas durante a execução de um *script*, enquanto uma base de dados guarda a informação entre pedidos diferentes, e entre utilizadores diferentes.

8.2. Comandos SQL

SQL (*Structured Query Language*) é a linguagem usada para interagir com um DBMS relacional. Os seus comandos dividem-se em quatro grupos principais:

- **DQL** (*Data Query Language*): consultas que obtêm dados, com o comando **SELECT**;
- **DML** (*Data Manipulation Language*): inserir, alterar e remover dados, com os comandos **INSERT**, **UPDATE** e **DELETE**;
- **DDL** (*Data Definition Language*): criar e alterar estruturas da base de dados, com os comandos **CREATE**, **ALTER** e **DROP**;
- **DCL** (*Data Control Language*): permissões e controlo de acesso, com os comandos **GRANT** e **REVOKE**.

Um comando SQL segue uma estrutura previsível: uma palavra-chave (**SELECT**, **UPDATE**, **INSERT**, **DELETE**), cláusulas que refinam a instrução (**FROM**, **WHERE**, **ORDER BY**, **VALUES**), expressões e condições (campos, valores e comparações, como *idade > 18*), e um ponto e vírgula final:

```
SELECT firstname, lastname FROM MyGuests WHERE id = 3;
```

Um comando de atualização junta estas mesmas partes de forma um pouco diferente, comando mais tabela alvo, a ação de alterar valores, e uma condição que filtra que registos são afetados:

```
UPDATE Employees SET Salary = Salary * 1.02 WHERE Country = 'USA';
```

8.3. A Arquitetura em Três Camadas e o Papel do PHP

Retomando o Secção 2.1.3, uma aplicação web organiza-se tipicamente em três camadas distintas: o cliente (*browser*), que mostra páginas e envia pedidos; o servidor, que processa a lógica da aplicação; e o DBMS, que guarda e gere os dados. Separar estas camadas aumenta a segurança, o desempenho e a organização da aplicação.

Vale a pena seguir, passo a passo, como circula a informação neste modelo, com um exemplo concreto:

1. O cliente envia um pedido, por exemplo `GET /produtos?categoria=livros`;
2. O servidor (PHP) interpreta o pedido e gera a instrução SQL correspondente: `SELECT * FROM produtos WHERE categoria = 'livros'`;
3. O DBMS (MySQL) executa essa instrução e devolve os resultados ao servidor;
4. O servidor insere esses dados num modelo (*template*) HTML;
5. O servidor devolve ao *browser* a página final, já preenchida com os dados pedidos.

Cada camada cumpre, assim, a sua própria missão: o *browser* pede, o servidor pensa, o DBMS guarda e responde, o servidor monta a página, e o *browser* mostra o resultado final ao utilizador. O PHP vive inteiramente na camada do servidor, sendo o responsável por receber os pedidos do cliente, construir as instruções SQL, comunicar com o DBMS, processar os resultados devolvidos, e gerar o HTML final enviado ao *browser*: é a ponte entre o pedido do utilizador e os dados guardados na base de dados.

8.4. Ligar o PHP ao MySQL

O PHP liga-se a uma base de dados MySQL através da extensão `mysqli` (*MySQL Improved*), que suporta a ligação ao MySQL, permite executar consultas, *prepared statements* e transações, e pode ser usada em dois estilos: procedural ou orientado a objetos.

No estilo procedural, `mysqli_connect()` cria a ligação e devolve `false` caso esta falhe, uma versão simples e direta, adequada a quem está a começar:

```
<?php
$conexao = mysqli_connect("localhost", "utilizador", "password", "nome_bd");

if (!$conexao) {
    die("Ligação falhou: " . mysqli_connect_error());
}
```

No estilo orientado a objetos, `new mysqli()` cria um objeto de ligação, e os erros ficam expostos na propriedade `connect_error` desse objeto. É o estilo mais moderno, e o mais usado na prática:

```
<?php
$conexao = new mysqli("localhost", "utilizador", "password", "nome_bd");

if ($conexao->connect_error) {
    die("Ligação falhou: " . $conexao->connect_error);
}
```

8.5. Criar Bases de Dados e Tabelas

Criar a base de dados e criar as tabelas são dois passos distintos: primeiro cria-se o “contentor” (a base de dados), só depois as estruturas internas (as tabelas). Tal como qualquer outra instrução SQL, ambos podem ser feitos a partir do PHP, através do método `query()`, que envia o comando SQL ao MySQL e devolve sucesso ou erro:

```
<?php
$conexao->query("CREATE DATABASE IF NOT EXISTS loja");
$conexao->select_db("loja");

$conexao->query("
    CREATE TABLE IF NOT EXISTS contactos (
        id INT AUTO_INCREMENT PRIMARY KEY,
        nome VARCHAR(100) NOT NULL,
        email VARCHAR(100) NOT NULL,
        criado_em DATETIME DEFAULT CURRENT_TIMESTAMP
    )
");
```

Vale a pena perceber o significado de cada parte desta definição de tabela:

- **PRIMARY KEY (PK)**: identifica, de forma inequívoca, cada registo da tabela; nenhuma tabela relacional bem desenhada deve existir sem uma;
- **AUTO_INCREMENT**: faz o valor deste campo aumentar automaticamente em uma unidade a cada novo registo inserido, sem que o programador precise de o calcular;
- **NOT NULL**: obriga a que este campo tenha sempre um valor, impedindo que fique vazio;
- **DEFAULT CURRENT_TIMESTAMP**: se não for indicado um valor explícito para este campo ao inserir um registo, o MySQL coloca automaticamente a data e hora atuais.

Na prática, criar a base de dados e as tabelas a partir do PHP é um passo inicial e raro numa aplicação real: a estrutura da base de dados de uma aplicação já em produção é, tipicamente, criada e alterada uma única vez, através de ferramentas próprias de administração, não a cada execução do *script*.

8.6. Operações Fundamentais: CRUD em SQL

As quatro operações fundamentais sobre dados persistentes, **CRUD** (*Create, Read, Update, Delete*), correspondem diretamente a quatro comandos SQL.

Inserir um novo registo (*Create*), com **INSERT**:

```
<?php
```

```
$conexao->query("INSERT INTO contactos (nome, email) VALUES ('Ana Silva', 'ana@exemplo.pt
```

Ler dados existentes (*Read*), com **SELECT**. A propriedade `num_rows` do resultado indica quantas linhas foram devolvidas, e o método `fetch_assoc()` devolve cada linha como um array associativo, tal como visto no Secção 3.6.3:

```
<?php
```

```
$resultado = $conexao->query("SELECT * FROM contactos");
```

```
echo $resultado->num_rows . " contactos encontrados";
```

```
while ($linha = $resultado->fetch_assoc()) {
```

```
    echo $linha["nome"] . " - " . $linha["email"] . "<br>";
```

```
}
```

Esta última construção, um **while** a percorrer os resultados linha a linha, é também a forma habitual de gerar uma tabela HTML dinamicamente a partir dos dados devolvidos pela base de dados, transformando cada linha da base de dados numa linha `<tr>` da página.

Atualizar (*Update*) e **apagar** (*Delete*) registos existentes, com **UPDATE** e **DELETE**:

```
<?php
```

```
$conexao->query("UPDATE contactos SET email = 'novo@exemplo.pt' WHERE id = 3");
```

```
$conexao->query("DELETE FROM contactos WHERE id = 3");
```

Tanto o **UPDATE** como o **DELETE** **exigem sempre uma cláusula WHERE**; esquecer-la é um dos erros mais graves e mais comuns que se pode cometer com uma base de dados, porque, sem uma condição que filtre os registos afetados, a instrução aplica-se a **todas** as linhas da tabela.

8.7. Prepared Statements

Todos os exemplos acima, com valores fixos escritos diretamente no código, servem para perceber o fluxo básico, mas não são adequados a uma aplicação real. Construir uma instrução SQL concatenando diretamente valores vindos do utilizador, como os de `$_POST`, tem vários problemas: mistura dados com código, é vulnerável a um ataque de injeção de SQL (tema aprofundado no Secção 9.1), é mais propenso a erros de sintaxe, e é difícil de manter.

A alternativa correta é usar *prepared statements*, que separam a instrução SQL, fixa, dos valores, que só são inseridos depois. Funcionam em duas fases: o servidor prepara a instrução, com pontos de interrogação (?) no lugar dos valores, e só depois o PHP envia os dados, de forma separada e segura. As vantagens são proteção contra injeção de SQL, melhor desempenho quando a mesma instrução é executada várias vezes, e um código mais limpo:

```
<?php
```

```
$stmt = $conexao->prepare(
```

```
    "INSERT INTO contactos (nome, email) VALUES (?, ?)"
```

```
);
```

```
$stmt->bind_param("ss", $nome, $email); // "ss" = dois parâmetros string
```

```
$nome = $_POST["nome"];
$email = $_POST["email"];
$stmt->execute();
```

O `bind_param()` precisa de saber o tipo de cada valor associado, indicado por uma letra:

- **s:** *string*;
- **i:** *integer*;
- **d:** *double* (números com casas decimais);
- **b:** *blob* (dados binários, como imagens ou ficheiros).

Para consultar dados com *prepared statements*, o padrão é semelhante, terminando com `get_result()` para obter as linhas devolvidas:

```
<?php
$stmt = $conexao->prepare("SELECT nome, email FROM contactos WHERE id = ?");
$stmt->bind_param("i", $id); // "i" = um parâmetro integer
$id = $_GET["id"];
$stmt->execute();

$resultado = $stmt->get_result();
while ($linha = $resultado->fetch_assoc()) {
    echo $linha["nome"] . " - " . $linha["email"] . "<br>";
}
```

O mesmo princípio aplica-se ao `UPDATE` e ao `DELETE`: usar *prepared statements* torna-os seguros, porque os valores nunca são interpretados como parte do próprio comando SQL, o que evita tanto a injeção de SQL como alterações acidentais.

8.8. Erros e Boas Práticas

Trabalhar com bases de dados implica, inevitavelmente, lidar com erros. Os mais comuns dividem-se em três categorias:

- **Erros de ligação:** credenciais incorretas, o servidor MySQL estar desligado, ou a base de dados indicada não existir;
- **Erros de SQL:** sintaxe inválida, nomes de tabelas ou colunas errados, falta de aspas ou vírgulas na instrução;
- **Erros de execução no PHP:** usar variáveis não definidas, ou chamar `fetch_assoc()` sobre um resultado vazio.

Durante o desenvolvimento, é essencial ativar a exibição de todos os erros e avisos do PHP, algo que nunca deve ficar ativo num ambiente de produção, por razões de segurança:

```
<?php
error_reporting(E_ALL);
ini_set("display_errors", 1);
```

O `mysqli` explica sempre o que falhou, bastando ler a informação correta: `connect_error` mostra problemas na própria ligação, e `error` mostra problemas vindos do MySQL relativos a uma consulta específica. Algumas boas práticas gerais, a manter sempre presentes ao trabalhar com bases de dados em PHP:

- Usar *prepared statements* sempre que houver dados vindos do utilizador;
- Validar e sanitizar as entradas antes de as usar;
- Separar claramente código PHP, SQL e HTML;
- Mostrar erros durante o desenvolvimento, mas ocultá-los em produção;
- Testar sempre com dados reais e também com dados propositadamente maliciosos.

8.9. Uma Aplicação CRUD Completa

Juntando tudo o que foi visto neste capítulo, uma pequena aplicação de gestão de contactos, sobre uma tabela `contacts`, distribui-se tipicamente por vários ficheiros PHP, cada um responsável por uma operação:

- `config.php`: estabelece a ligação à base de dados, e é incluído, com `require`, no início de todos os outros ficheiros, evitando repetir o código de ligação em cada um;
- `index.php`: lista todos os contactos existentes (*Read*), e liga a todas as outras operações;
- `add.php` e `insert.php`: `add.php` apresenta um formulário HTML, que envia os seus dados por POST para `insert.php`, o qual, com um *prepared statement*, insere o novo registo (*Create*) e redireciona de volta para `index.php`;
- `edit.php` e `update.php`: `edit.php` apresenta um formulário pré-preenchido com os dados de um registo existente, e `update.php` grava as alterações (*Update*);
- `delete.php`: remove um registo existente (*Delete*).

Um detalhe merece atenção especial no `index.php`: cada `link` para editar ou apagar um contacto transporta o `id` desse contacto na própria URL, por exemplo `edit.php?id=7`, para que a página seguinte saiba exatamente a que registo se refere. Este `id`, por vir de um GET simples, sem dados sensíveis nem efeitos colaterais até ser efetivamente processado, não exige, aqui, um *prepared statement* na listagem; já em `edit.php`, ao ler esse mesmo `id` do GET para o incluir de novo no formulário, este tem de ser colocado num campo `hidden` do formulário, para que, ao ser submetido por POST para `update.php`, o `id` do registo a alterar viaje junto com os restantes dados.

O fluxo completo da aplicação pode resumir-se assim: o utilizador navega entre páginas simples e independentes a partir de `index.php`; cada operação, `add`, `edit` ou `delete`, comunica com a base de dados através de `insert.php`, `update.php` ou diretamente em `delete.php`; e, depois de gravada a alteração, a aplicação redireciona sempre de volta para `index.php`, fechando o ciclo.

Este pequeno projeto reúne, na prática, praticamente tudo o que foi visto até este ponto da UC: HTML e formulários do lado do cliente, supervariáveis para receber os dados submetidos, *prepared statements* para os gravar e ler em segurança, e uma aplicação organizada em várias páginas simples, cada uma com uma única responsabilidade, o padrão sobre o qual assenta a maioria das aplicações web dinâmicas.

Referências

- Bytebase. 2026. «Drizzle ORM vs Prisma: Which TypeScript ORM Should You Use in 2026?». 2026. <https://www.bytebase.com/blog/drizzle-vs-prisma/>.
- DreamFactory. 2025. «45 SOAP to REST Conversion Statistics Every Enterprise Should Know». 2025. <https://www.dreamfactory.com/hub/soap-to-rest-conversion-statistics>.
- Google. s.d. «Cloud Firestore Data Model». <https://firebase.google.com/docs/firestore/data-model>.
- JetBrains. 2025. «The State of PHP 2025». 2025. <https://blog.jetbrains.com/phpstorm/2025/10/state-of-php-2025/>.
- Mapsoft. s.d. «Adobe XD End of Life: Status, Timeline & Alternatives». <https://mapsoft.com/adobe-xd-end-of-life/>.
- «Native CSS Nesting: Sass Syntax With No Build Step (And One Big Trap)». 2025. 2025. <https://www.w3tweaks.com/css/css-nesting-explained/>.
- OWASP Foundation. 2026. «OWASP Top 10:2025». 2026. <https://owasp.org/Top10/2025/>.
- Patchstack. 2025. «Patchstack’s 2025 WordPress Security Report: Mid-Year Vulnerability Breakdown». 2025. <https://patchstack.com/whitepaper/2025-mid-year-vulnerability-report/>.
- Reenskaug, Trygve. 2003. «MVC: Xerox PARC 1978-79». 2003. <https://citeseerx.ist.psu.edu/document?repid=rep1&type=pdf&doi=4ef90a7b9c1b1cd02acf273694e4059a70c7d198>.
- «State of JavaScript 2025: Front-end Frameworks». 2025. 2025. <https://2025.stateofjs.com/en-US/libraries/front-end-frameworks/>.
- «Top CSS Frameworks in 2025: What Devs Actually Use». 2025. 2025. <https://tailkits.com/blog/popular-css-frameworks/>.

9. Segurança em Aplicações Web

9.1. Vulnerabilidades no Código Próprio

A referência mais usada para classificar vulnerabilidades de aplicações web é o OWASP Top 10, uma lista dos riscos de segurança mais críticos, mantida pela *Open Worldwide Application Security Project* e atualizada periodicamente com base na análise de dezenas de milhares de vulnerabilidades reais (OWASP Foundation 2026). Três destas vulnerabilidades são particularmente relevantes para uma aplicação PHP como as desenvolvidas nesta UC:

9.1.1. SQL Injection

O SQL Injection ocorre quando dados fornecidos pelo utilizador são inseridos diretamente numa instrução SQL, permitindo que um atacante altere o significado dessa instrução. Considere-se um formulário de autenticação implementado de forma insegura, por concatenação direta:

```
<?php
$utilizador = $_POST["utilizador"];
$password = $_POST["password"];

$sql = "SELECT * FROM utilizadores WHERE utilizador = '$utilizador' AND password = '$password'";
$resultado = $conexao->query($sql);
```

Se um atacante escrever, no campo utilizador, o texto ' OR '1'='1' --, a instrução SQL efetivamente executada passa a ser:

```
SELECT * FROM utilizadores WHERE utilizador = ' OR '1'='1' -- ' AND password = '...'
```

Como '1'='1' é sempre verdadeiro, e o -- transforma o resto da linha num comentário SQL, ignorando a verificação da password, esta consulta devolve o primeiro utilizador da tabela, permitindo ao atacante iniciar sessão sem conhecer nenhuma password válida. É evitada usando sempre *prepared statements*, como visto no Secção 8.7, que tratam o conteúdo de \$_POST sempre como um valor, nunca como parte do comando SQL, independentemente do que esse valor contenha.

9.1.2. XSS (Cross-Site Scripting)

O XSS ocorre quando dados fornecidos por um utilizador são apresentados a outros utilizadores sem serem devidamente tratados, permitindo a um atacante injetar código JavaScript que corre no *browser* de quem visualiza essa página. Considere-se uma caixa de comentários que mostra o texto submetido diretamente:

```
<?php
echo "<p>" . $_POST["comentario"] . "</p>";
```

Se um atacante submeter, como comentário, o texto `<script>document.location='http://atacante.com/'`, esse código JavaScript passa a fazer parte da página tal como qualquer outro, e é executado automaticamente no *browser* de qualquer outro utilizador que visite essa página, podendo, por exemplo, enviar os seus *cookies* de sessão para um servidor controlado pelo atacante. É evitada escapando sempre conteúdo gerado por utilizadores antes de o apresentar em HTML, com a função `htmlspecialchars()`, que transforma caracteres especiais de HTML, como `<` e `>`, nas suas versões inofensivas:

```
<?php
echo "<p>" . htmlspecialchars($_POST["comentario"]) . "</p>";
```

Desta forma, o texto do atacante é mostrado literalmente na página, como texto visível, e não interpretado como código.

9.1.3. CSRF (Cross-Site Request Forgery)

O CSRF ocorre quando um atacante induz o *browser* de um utilizador já autenticado a enviar, sem que este se aperceba, um pedido a uma aplicação onde tem sessão iniciada, explorando o facto de o *browser* enviar automaticamente os *cookies* de sessão em qualquer pedido a esse domínio, tal como visto no Secção 7.7.1. Imagine-se uma aplicação bancária com uma transferência implementada como um simples pedido GET:

```
GET /transferir?destino=1234&valor=500
```

Se um utilizador, autenticado nesse banco, visitar, noutra separador do *browser*, uma página maliciosa que contenha ``, o *browser* tenta carregar essa “imagem”, enviando automaticamente o pedido ao banco, junto com o *cookie* de sessão válido do utilizador, sem que este tenha clicado em nada. É evitada de duas formas complementares: nunca implementar operações que alterem dados como um simples GET, tal como já recomendado no Secção 7.3.3, e incluir, em cada formulário sensível, um token único e imprevisível, gerado pelo servidor e verificado antes de processar o pedido, que uma página maliciosa, alojada noutra domínio, não tem forma de conhecer nem reproduzir.

Em todos estes casos, o princípio subjacente é o mesmo: nunca confiar em dados vindos do cliente, seja de um formulário, de um parâmetro na URL ou de um *cookie*, sem primeiro os validar e tratar adequadamente.

9.2. Vulnerabilidades em Dependências de Terceiros

A superfície de ataque de um sítio web não se limita ao código escrito diretamente pelo seu programador. Grande parte dos sítios web atuais é construída sobre sistemas de gestão de conteúdos, como o **WordPress**, estendidos através de extensões (*plugins*) desenvolvidas por terceiros, muitas vezes por equipas pequenas ou até um único programador independente. Segundo o relatório de segurança da Patchstack referente a 2025, 91% das vulnerabilidades encontradas no ecossistema WordPress nesse ano estavam localizadas em *plugins*, não no núcleo do **WordPress**, e mais de metade das infeções registadas resultaram de *plugins* desatualizados (Patchstack 2025).

Esta realidade levou a atualização mais recente do OWASP Top 10 a introduzir uma categoria dedicada a falhas na cadeia de fornecimento de *software* (*Software Supply Chain Failures*), reconhecendo que a segurança de uma aplicação depende também da segurança de todas as bibliotecas, *frameworks* e extensões de que esta depende, e não apenas do código escrito pela própria equipa (OWASP Foundation 2026). É por esta razão que, na prática, os serviços de alojamento suspendem por vezes o acesso a um sítio web até que as suas dependências vulneráveis sejam atualizadas: a vulnerabilidade pode não estar em nenhuma linha de código escrita pelo dono do sítio, mas o risco que representa é igualmente real.

9.3. Boas Práticas

Algumas práticas simples reduzem significativamente o risco de segurança de uma aplicação web:

- **Atualizações regulares:** manter o *software* de base (o próprio PHP, o servidor web, o sistema de gestão de bases de dados) e todas as dependências e extensões de terceiros atualizadas para a versão mais recente, já que muitas vulnerabilidades só são exploradas depois de serem publicamente conhecidas;
- **Princípio do menor privilégio:** cada componente de uma aplicação, e cada utilizador da base de dados, deve ter apenas as permissões estritamente necessárias ao seu funcionamento, nunca acesso total por comodidade;
- **Validação e sanitização de dados:** tratar sempre com desconfiança qualquer dado que venha do exterior da aplicação, validando o seu formato e sanitizando o seu conteúdo antes de o usar, seja numa consulta à base de dados, seja numa página apresentada a outro utilizador.

Referências

- Bytebase. 2026. «Drizzle ORM vs Prisma: Which TypeScript ORM Should You Use in 2026?» 2026. <https://www.bytebase.com/blog/drizzle-vs-prisma/>.
- DreamFactory. 2025. «45 SOAP to REST Conversion Statistics Every Enterprise Should Know». 2025. <https://www.dreamfactory.com/hub/soap-to-rest-conversion-statistics>.
- Google. s.d. «Cloud Firestore Data Model». <https://firebase.google.com/docs/firestore/data-model>.
- JetBrains. 2025. «The State of PHP 2025». 2025. <https://blog.jetbrains.com/phpstorm/2025/10/state-of-php-2025/>.
- Mapsoft. s.d. «Adobe XD End of Life: Status, Timeline & Alternatives». <https://mapsoft.com/adobe-xd-end-of-life/>.

- «Native CSS Nesting: Sass Syntax With No Build Step (And One Big Trap)». 2025. 2025. <https://www.w3tweaks.com/css/css-nesting-explained/>.
- OWASP Foundation. 2026. «OWASP Top 10:2025». 2026. <https://owasp.org/Top10/2025/>.
- Patchstack. 2025. «Patchstack’s 2025 WordPress Security Report: Mid-Year Vulnerability Breakdown». 2025. <https://patchstack.com/whitepaper/2025-mid-year-vulnerability-report/>.
- Reenskaug, Trygve. 2003. «MVC: Xerox PARC 1978-79». 2003. <https://citeseerx.ist.psu.edu/document?repid=rep1&type=pdf&doi=4ef90a7b9c1b1cd02acf273694e4059a70c7d198>.
- «State of JavaScript 2025: Front-end Frameworks». 2025. 2025. <https://2025.stateofjs.com/en-US/libraries/front-end-frameworks/>.
- «Top CSS Frameworks in 2025: What Devs Actually Use». 2025. 2025. <https://tailkits.com/blog/popular-css-frameworks/>.

10. Model-View-Controller (MVC)

10.1. Porquê Separar Responsabilidades?

A mini aplicação CRUD construída no Secção 8.9, com ficheiros como `index.php`, `add.php` ou `edit.php`, ilustra bem um problema muito concreto: em cada um desses ficheiros misturam-se, no mesmo lugar, a consulta à base de dados, a validação dos dados recebidos e o HTML apresentado ao utilizador. Numa aplicação pequena, com uma única tabela, isto é gerível. À medida que uma aplicação cresce, com mais entidades, mais regras de negócio e mais páginas, este estilo de código torna-se cada vez mais difícil de manter: uma alteração simples ao aspeto visual de uma página obriga a mexer no mesmo ficheiro que contém a lógica de acesso aos dados, e um erro numa consulta SQL pode, sem se perceber porquê, quebrar também a apresentação da página. É exatamente este problema que o padrão MVC, apresentado no Secção 2.1.2, resolve, separando a aplicação em Model, View e Controller.

10.2. Refazer o CRUD com MVC

Aplicar o MVC à mini aplicação de contactos significa reorganizar o código, mantendo o mesmo comportamento, mas separando-o em ficheiros com responsabilidades bem definidas:

- **Model** (`Contacto.php`): concentra toda a lógica de acesso à base de dados relativa a contactos, com funções como `listarContactos()`, `criarContacto($nome, $email)`, `atualizarContacto($id, $nome, $email)` e `apagarContacto($id)`. Nenhuma destas funções produz HTML; devolvem apenas dados.
- **View** (`views/lista.php`, `views/formulario.php`): concentram apenas HTML, com pequenos trechos de PHP para apresentar os dados que lhes são fornecidos, sem qualquer acesso direto à base de dados.
- **Controller** (`ContactoController.php`): recebe o pedido do utilizador, decide que função do *Model* chamar consoante a ação pedida, seja listar, criar, editar ou apagar, e escolhe a *View* apropriada para mostrar o resultado.

```

<?php
// ContactoController.php (excerto)
require "Contacto.php";

function acaoListar() {
    $contactos = listarContactos();
    require "views/lista.php"; // a View usa $contactos
}

function acaoCriar() {
    if ($_SERVER["REQUEST_METHOD"] === "POST") {
        criarContacto($_POST["nome"], $_POST["email"]);
        header("Location: index.php");
        exit;
    }
    require "views/formulario.php";
}

```

O comportamento final, para quem usa a aplicação, é o mesmo de antes. O que mudou foi inteiramente interno: quem altera o aspeto visual mexe apenas nas *Views*, quem altera as regras de negócio mexe apenas no *Model*, e o *Controller* liga as duas partes sem depender dos detalhes internos de nenhuma delas.

10.3. Regras de Negócio no Model

O *Model* não se limita a executar consultas SQL: é também o lugar natural para as regras de negócio da aplicação, a lógica que determina se uma operação é válida, independentemente de como os dados chegaram até ali. Considere-se a validação de um endereço de email antes de criar um novo contacto:

```

<?php
// Contacto.php (excerto)
function criarContacto($nome, $email) {
    if (!filter_var($email, FILTER_VALIDATE_EMAIL)) {
        return ["sucesso" => false, "erro" => "Email inválido"];
    }

    global $conexao;
    $stmt = $conexao->prepare("INSERT INTO contactos (nome, email) VALUES (?, ?)");
    $stmt->bind_param("ss", $nome, $email);
    $stmt->execute();

    return ["sucesso" => true];
}

```

Esta validação faz sentido no *Model*, e não no *Controller* ou na *View*, porque é uma regra sobre os dados em si, “um contacto tem de ter um email válido”, que se aplica sempre que um contacto é criado, independentemente de o pedido ter vindo de um formulário web, de uma futura API, ou de um script de importação em lote. Colocar esta verificação no *Controller*,

por comparação, obrigaria a repeti-la em cada novo ponto de entrada que viesse a criar contactos.

10.4. O Controlador Frontal (Front Controller)

A mini aplicação original tinha um ficheiro PHP por cada ação: `index.php`, `add.php`, `edit.php`, `delete.php`. Uma alternativa comum, sobretudo à medida que o número de ações cresce, é o padrão de *controlador frontal* (*front controller*): em vez de vários pontos de entrada, existe um único ficheiro, normalmente também chamado `index.php`, que recebe todos os pedidos e decide, com base num parâmetro, que ação do *Controller* deve executar:

```
<?php
// index.php como controlador frontal
require "ContactoController.php";

$acao = $_GET["acao"] ?? "listar";

switch ($acao) {
    case "criar":
        acaoCriar();
        break;
    case "editar":
        acaoEditar();
        break;
    default:
        acaoListar();
}
```

Este padrão centraliza decisões que, de outra forma, teriam de ser repetidas em cada ficheiro, como verificar se o utilizador tem sessão iniciada antes de permitir qualquer ação. É também o mecanismo que a generalidade dos *frameworks* de desenvolvimento web, tema do próximo capítulo, usa por trás do seu sistema de rotas.

Referências

- Bytebase. 2026. «Drizzle ORM vs Prisma: Which TypeScript ORM Should You Use in 2026?» 2026. <https://www.bytebase.com/blog/drizzle-vs-prisma/>.
- DreamFactory. 2025. «45 SOAP to REST Conversion Statistics Every Enterprise Should Know». 2025. <https://www.dreamfactory.com/hub/soap-to-rest-conversion-statistics>.
- Google. s.d. «Cloud Firestore Data Model». <https://firebase.google.com/docs/firestore/data-model>.
- JetBrains. 2025. «The State of PHP 2025». 2025. <https://blog.jetbrains.com/phpstorm/2025/10/state-of-php-2025/>.
- Mapsoft. s.d. «Adobe XD End of Life: Status, Timeline & Alternatives». <https://mapsoft.com/adobe-xd-end-of-life/>.
- «Native CSS Nesting: Sass Syntax With No Build Step (And One Big Trap)». 2025. 2025. <https://www.w3tweaks.com/css/css-nesting-explained/>.
- OWASP Foundation. 2026. «OWASP Top 10:2025». 2026. <https://owasp.org/Top10/2025/>.

Referências

- Patchstack. 2025. «Patchstack’s 2025 WordPress Security Report: Mid-Year Vulnerability Breakdown». 2025. <https://patchstack.com/whitepaper/2025-mid-year-vulnerability-report/>.
- Reenskaug, Trygve. 2003. «MVC: Xerox PARC 1978-79». 2003. <https://citeseerx.ist.psu.edu/document?repid=rep1&type=pdf&doi=4ef90a7b9c1b1cd02acf273694e4059a70c7d198>.
- «State of JavaScript 2025: Front-end Frameworks». 2025. 2025. <https://2025.stateofjs.com/en-US/libraries/front-end-frameworks/>.
- «Top CSS Frameworks in 2025: What Devs Actually Use». 2025. 2025. <https://tailkits.com/blog/popular-css-frameworks/>.

11 Frameworks de Desenvolvimento Web

11.1. O Que um Framework Automatiza

Ao longo desta UC, construíram-se, ficheiro a ficheiro, os elementos básicos de uma aplicação PHP: ligação à base de dados, um controlador frontal a decidir que ação executar, e a separação entre *Model*, *View* e *Controller*, vista no capítulo anterior. Um *framework* de desenvolvimento web como o **Laravel**, hoje o mais usado no ecossistema PHP (JetBrains 2025), automatiza precisamente este tipo de trabalho repetitivo, oferecendo já resolvidas, e testadas por uma grande comunidade de utilizadores, partes que, de outra forma, cada programador teria de construir à sua maneira em cada novo projeto:

- **Sistema de rotas:** em vez do `switch` manual construído no Secção 10.4, um *framework* permite declarar rotas de forma direta, associando um endereço a uma função do *Controller*;
- **ORM** (*Object-Relational Mapping*): em vez de escrever SQL à mão com `mysqli`, como no Secção 8.7, um ORM representa cada tabela como uma classe PHP, e cada linha como um objeto, permitindo interagir com a base de dados através de código PHP em vez de instruções SQL diretas;
- **Motor de *templates*:** em vez de misturar PHP e HTML diretamente nas *Views*, um motor de *templates* oferece uma sintaxe mais limpa para gerar HTML dinâmico, com funcionalidades como herança de *layouts*, para não repetir cabeçalho e rodapé em cada página;
- **Migrações:** ficheiros que descrevem, em código, a estrutura da base de dados, permitindo criar, alterar e partilhar essa estrutura entre programadores sem depender de instruções `CREATE TABLE` escritas manualmente por cada um.

11.2. O Laravel

O Laravel inclui, entre outros, o **Eloquent**, o seu ORM nativo, e o **Blade**, o seu motor de *templates*. Uma consulta que, com `mysqli`, exigiria escrever a instrução SQL explicitamente, escreve-se, em Eloquent, como uma chamada de método sobre uma classe que representa a tabela:

```
// Com mysqli (capítulo 8)
$stmt = $conexao->prepare("SELECT * FROM contactos WHERE id = ?");
$stmt->bind_param("i", $id);
$stmt->execute();

// Com Eloquent (Laravel)
$contacto = Contacto::find($id);
```

O sistema de rotas do Laravel substitui diretamente o **switch** manual do controlador frontal construído no Secção 10.4, associando um endereço a uma função de forma declarativa:

```
// routes/web.php
Route::get('/contactos', [ContactoController::class, 'listar']);
Route::post('/contactos', [ContactoController::class, 'criar']);
Route::get('/contactos/{id}/editar', [ContactoController::class, 'editar']);
```

As migrações substituem, por sua vez, as instruções **CREATE TABLE** escritas à mão no Secção 8.5, descrevendo a estrutura da tabela em código PHP, versionável e partilhável entre toda a equipa:

```
// database/migrations/create_contactos_table.php
Schema::create('contactos', function (Blueprint $table) {
    $table->id(); // equivalente a INT AUTO_INCREMENT PRIMARY KEY
    $table->string('nome');
    $table->string('email');
    $table->timestamps(); // equivalente a criado_em DATETIME DEFAULT CURRENT_TIMESTAMP
});
```

O Laravel gera, também, automaticamente, grande parte da estrutura de pastas que foi construída manualmente ao longo desta UC: uma pasta **app/Models** para as classes do *Model*, uma pasta **resources/views** para as *Views* em Blade, e uma pasta **routes** para declarar as rotas da aplicação. A curva de aprendizagem inicial de um *framework* como o Laravel é mais exigente do que escrever PHP simples, porque exige compreender as suas convenções próprias, mas paga-se rapidamente à medida que uma aplicação cresce, evitando que cada equipa reinvente, à sua maneira, os mesmos mecanismos de ligação à base de dados, autenticação ou geração de páginas.

11.3. Outros Frameworks

O Laravel não é o único *framework* PHP: o **Symfony**, mais modular e usado sobretudo em contextos empresariais, é o segundo mais adotado (JetBrains 2025), e partes dele são, de resto, usadas internamente pelo próprio Laravel. Fora do universo PHP, o mesmo princípio aplica-se a outras linguagens do lado do servidor, como o **Django** em Python ou o **Express** em JavaScript, referidos no Secção 1.6.2: cada um resolve o mesmo tipo de problema, rotas, acesso a dados, geração de páginas, com convenções e ferramentas próprias da sua linguagem.

Referências

- Bytebase. 2026. «Drizzle ORM vs Prisma: Which TypeScript ORM Should You Use in 2026?». 2026. <https://www.bytebase.com/blog/drizzle-vs-prisma/>.
- DreamFactory. 2025. «45 SOAP to REST Conversion Statistics Every Enterprise Should Know». 2025. <https://www.dreamfactory.com/hub/soap-to-rest-conversion-statistics>.
- Google. s.d. «Cloud Firestore Data Model». <https://firebase.google.com/docs/firestore/data-model>.
- JetBrains. 2025. «The State of PHP 2025». 2025. <https://blog.jetbrains.com/phpstorm/2025/10/state-of-php-2025/>.
- Mapsoft. s.d. «Adobe XD End of Life: Status, Timeline & Alternatives». <https://mapsoft.com/adobe-xd-end-of-life/>.
- «Native CSS Nesting: Sass Syntax With No Build Step (And One Big Trap)». 2025. 2025. <https://www.w3tweaks.com/css/css-nesting-explained/>.
- OWASP Foundation. 2026. «OWASP Top 10:2025». 2026. <https://owasp.org/Top10/2025/>.
- Patchstack. 2025. «Patchstack’s 2025 WordPress Security Report: Mid-Year Vulnerability Breakdown». 2025. <https://patchstack.com/whitepaper/2025-mid-year-vulnerability-report/>.
- Reenskaug, Trygve. 2003. «MVC: Xerox PARC 1978-79». 2003. <https://citeseerx.ist.psu.edu/document?repid=rep1&type=pdf&doi=4ef90a7b9c1b1cd02acf273694e4059a70c7d198>.
- «State of JavaScript 2025: Front-end Frameworks». 2025. 2025. <https://2025.stateofjs.com/en-US/libraries/front-end-frameworks/>.
- «Top CSS Frameworks in 2025: What Devs Actually Use». 2025. 2025. <https://tailkits.com/blog/popular-css-frameworks/>.

A. Configuração do Ambiente de Desenvolvimento

A.1. Introdução

Este apêndice descreve, passo a passo, a configuração do ambiente de desenvolvimento apresentado no Secção 3.1, tanto por máquina virtual como por contentores. Há, para cada uma destas duas abordagens, dois cenários possíveis:

- **O professor fornece o ambiente já preparado:** uma máquina virtual já instalada e configurada, distribuída como ficheiro `.ova`, ou um conjunto de contentores já definidos, distribuídos como ficheiros de configuração ou como imagens já construídas. Este é o cenário mais comum, e o único que a maioria dos alunos precisa de seguir.
- **O aluno constrói o ambiente do zero:** instalando e configurando ele próprio o Apache, o MySQL e o PHP, ou escrevendo o seu próprio `Dockerfile` e `docker-compose.yml`. É útil para quem quer usar o mesmo ambiente num computador pessoal fora do contexto da aula, ou simplesmente compreender, em detalhe, do que é feito o ambiente que lhe é disponibilizado.

Nenhuma destas descrições substitui as instruções concretas que possam vir a ser dadas nas aulas práticas, que podem especificar versões, configurações de rede ou credenciais diferentes das aqui usadas como exemplo; o objetivo deste apêndice é apenas garantir que qualquer aluno consegue, por si só, chegar a um ambiente funcional, com o Apache, o PHP e o MySQL prontos a usar.

A.1.1. Sistemas Operativos dos Computadores dos Alunos

Antes de descarregar qualquer *software*, é importante saber qual das seguintes três variantes corresponde ao computador de cada aluno, já que tanto o `VirtualBox` como o `Docker Desktop` disponibilizam instaladores diferentes para cada uma:

- **Windows;**
- **macOS com processador Intel** (Mac mais antigos, anteriores a 2020/2021);
- **macOS com Apple Silicon** (Mac com processador M1, M2, M3 ou M4, a generalidade dos modelos vendidos a partir de 2020/2021).

Num Mac, esta informação obtém-se no menu **Apple** (o ícone no canto superior esquerdo do ecrã), em *Sobre Este Mac*: se o campo **Chip** mostrar **Apple M1, M2, M3 ou M4**, trata-se de Apple Silicon; se mostrar **Processador Intel**, trata-se de um Mac Intel. Escolher o instalador errado é um dos erros mais comuns nesta fase, e resulta tipicamente numa aplicação que não abre, ou que abre mas falha ao iniciar qualquer máquina virtual ou contentor.

A.1.2. O phpMyAdmin

O **phpMyAdmin** é uma ferramenta gratuita de administração do **MySQL** através do *browser*, escrita, ela própria, em **PHP**. Em vez de escrever instruções **SQL** na linha de comandos para ver o conteúdo de uma tabela, criar uma base de dados, ou verificar se uma linha foi corretamente inserida, o **phpMyAdmin** mostra tudo isto numa interface visual, o que o torna particularmente útil enquanto se aprende a trabalhar com bases de dados. Nem a máquina virtual, nem os contentores, precisam obrigatoriamente de o incluir, o **MySQL** funciona perfeitamente sem ele, mas, quando disponível, é frequentemente a forma mais rápida de confirmar visualmente que uma aplicação **PHP** está, de facto, a gravar e a ler os dados corretos. Este apêndice indica, em cada uma das quatro secções seguintes, como aceder ao **phpMyAdmin** quando este é fornecido pelo professor, e como o instalar quando o ambiente é construído do zero.

A.2. Máquina Virtual Fornecida pelo Professor

A.2.1. Instalar o VirtualBox

O único *software* que cada aluno precisa de instalar, neste cenário, é o **VirtualBox**, disponível gratuitamente em [virtualbox.org/wiki/Downloads](https://www.virtualbox.org/wiki/Downloads), onde aparecem pacotes de instalação separados consoante o sistema operativo do computador: **Windows hosts**, **macOS / Intel hosts** e **macOS / Apple Silicon hosts**, correspondendo às três variantes descritas no Secção A.1.1. A máquina virtual em si, com o **Ubuntu** e toda a *stack* de desenvolvimento já instalados e configurados, é fornecida separadamente pelo professor, como um ficheiro **.ova**.

A.2.2. Importar o Ficheiro .ova

Com o **VirtualBox** instalado e o ficheiro **.ova** fornecido pelo professor guardado no computador, os passos para o pôr a funcionar são:

1. Abrir o **VirtualBox** e escolher **Ficheiro > Importar Appliance...** (ou o atalho **Ctrl+I**);
2. Selecionar o ficheiro **.ova** fornecido;
3. Rever as definições propostas para a máquina virtual (nome, memória RAM, número de processadores, localização do disco virtual), que já vêm preenchidas com os valores usados pelo professor, e que podem ser ajustadas, dentro dos limites do computador de cada aluno, antes de avançar;
4. Confirmar a importação e aguardar que o **VirtualBox** copie e descomprima o conteúdo do ficheiro **.ova**, um processo que pode demorar vários minutos, consoante o tamanho da máquina virtual e a velocidade do disco;
5. Concluída a importação, selecionar a máquina virtual na lista do **VirtualBox** e clicar em **Iniciar**.

As credenciais de acesso (nome de utilizador e password) dentro da máquina virtual, bem como o endereço e a porta onde a aplicação fica acessível a partir do *browser* do computador anfitrião (por exemplo, `http://localhost:8080`, caso a máquina virtual use reencaminhamento de portas em vez de acesso direto), são indicados pelo professor juntamente com o ficheiro `.ova`, já que podem variar de ano para ano.

A.2.3. Primeiros Testes

Depois de a máquina virtual arrancar, antes de começar a programar, vale a pena confirmar que tudo está a funcionar como esperado. Num terminal dentro da máquina virtual, verifica-se a versão de cada componente já instalado:

```
php -v
apache2 -v
mysql --version
```

De seguida, testa-se a ligação ao PHP através do *browser*: abrindo o endereço indicado pelo professor (dentro da máquina virtual, tipicamente `http://localhost`; a partir do computador anfitrião, consoante a configuração de rede da máquina virtual), deve aparecer uma página servida pelo Apache e processada pelo PHP. Por fim, testa-se a ligação à base de dados, seja diretamente no terminal:

```
mysql -u root -p
```

seja através de um pequeno *script* PHP como o apresentado no Secção A.5, que confirma que o PHP consegue efetivamente comunicar com o MySQL, e não apenas que os dois serviços estão, cada um por si, a correr.

Se a máquina virtual fornecida pelo professor incluir também o `phpMyAdmin`, uma ferramenta de administração do MySQL através do *browser*, referida no Secção A.1.2, é essa a forma mais simples de confirmar visualmente que a base de dados está acessível, sem ser necessário usar a linha de comandos: basta abrir `http://localhost/phpmyadmin` (ou o endereço indicado pelo professor) e autenticar-se com as credenciais do MySQL.

A.3. Contentores Fornecidos pelo Professor

A.3.1. Instalar o Docker

Tal como na opção da máquina virtual, o único *software* que cada aluno precisa de instalar, neste cenário, é o Docker. Em Windows e em macOS, instala-se o `Docker Desktop`, disponível em docker.com/products/docker-desktop, onde aparecem, tal como no `VirtualBox`, instaladores separados para `Mac with Apple silicon` e `Mac with Intel chip`, além do instalador para Windows; em Linux, instala-se diretamente o `Docker Engine`, seguindo as instruções em docs.docker.com/engine/install. Confirma-se a instalação, num terminal do computador anfitrião, com:

```
docker --version
docker compose version
```

A.3.2. Carregar e Arrancar os Contentores

O professor pode distribuir o ambiente de duas formas distintas:

- **Ficheiros de configuração** (`docker-compose.yml`, e um `Dockerfile` se aplicável): bastando colocá-los numa pasta e correr, aí dentro:

```
docker compose up -d --build
```

- **Imagens já construídas**, distribuídas como ficheiros `.tar`: neste caso, começa-se por carregar cada imagem para o Docker local, antes de a arrancar:

```
docker load -i imagem-apache-php.tar
docker load -i imagem-mysql.tar
docker compose up -d
```

Em qualquer dos casos, o resultado esperado é o mesmo referido no Secção 3.1.2: um contentor com o Apache e o PHP, e outro contentor, separado, com o MySQL, ambos a correr e a comunicar entre si.

A.3.3. Primeiros Testes

Com os contentores a correr, confirma-se primeiro que ambos estão ativos:

```
docker ps
```

devendo aparecer, na lista, um contentor para o Apache/PHP e outro para o MySQL, cada um com o seu próprio nome. Verifica-se, depois, a versão instalada em cada um, executando o comando diretamente dentro do contentor com `docker exec`:

```
docker exec -it <nome_do_contentor_web> php -v
docker exec -it <nome_do_contentor_db> mysql --version
```

Testa-se, de seguida, a ligação ao PHP através do *browser*, no endereço e porta indicados pelo professor (por exemplo, `http://localhost:8080`), e, por fim, a ligação à base de dados, seja diretamente a partir do contentor:

```
docker exec -it <nome_do_contentor_db> mysql -u root -p
```

seja através do pequeno *script* PHP apresentado no Secção A.5, tal como no cenário da máquina virtual.

Se o professor incluir também um contentor com o `phpMyAdmin`, referido no Secção A.1.2, este fica normalmente acessível num endereço próprio (por exemplo, `http://localhost:8081`), oferecendo uma forma visual, através do *browser*, de confirmar o conteúdo da base de dados, sem ser necessário `docker exec` para entrar no contentor do MySQL.

A.4. Construir o Ambiente a Partir do Zero

As duas secções seguintes destinam-se a quem não vai usar o ambiente fornecido pelo professor, e prefere construir, ele próprio, cada uma das duas abordagens descritas no Secção 3.1, desde a instalação do VirtualBox ou do Docker até ao Apache, ao MySQL e ao PHP prontos a usar.

A.4.1. Opção 1: Máquina Virtual com VirtualBox

A.4.1.1. Criar a Máquina Virtual

1. Instalar o VirtualBox, tal como descrito no Secção A.2.1;
2. Obter uma imagem .iso do Ubuntu, disponível em ubuntu.com/download (a versão *Desktop*, mais simples de usar para quem começa, ou a versão *Server*, mais leve, para quem já tem alguma experiência com a linha de comandos);
3. No VirtualBox, criar uma nova máquina virtual, indicando o tipo Linux e a versão Ubuntu (64-bit);
4. Atribuir à máquina virtual pelo menos 4 GB de memória RAM, e um disco virtual de tamanho dinamicamente alocado, com pelo menos 25 GB, para acomodar o sistema operativo, os pacotes instalados e os projetos desenvolvidos ao longo da UC;
5. Associar a imagem .iso do Ubuntu como suporte de arranque, e iniciar a máquina virtual para correr o instalador, indicando o idioma, o teclado, e criando um utilizador e uma password;
6. Concluída a instalação, reiniciar a máquina virtual e instalar as **Guest Additions** do VirtualBox (disponíveis no menu *Dispositivos* do próprio VirtualBox), que melhoram o desempenho gráfico e permitem, mais tarde, configurar pastas partilhadas com o computador anfitrião.

A.4.1.2. Instalar o Apache, o MySQL e o PHP

Com o Ubuntu instalado e a correr dentro da máquina virtual, abre-se um terminal e atualiza-se a lista de pacotes disponíveis antes de instalar qualquer coisa:

```
sudo apt update && sudo apt upgrade -y
```

Instala-se depois o servidor web Apache:

```
sudo apt install apache2 -y
```

Verifica-se que o Apache ficou a correr, e testa-se, no *browser* da própria máquina virtual, o endereço `http://localhost`, que deve mostrar a página de boas-vindas por omissão do Apache:

```
sudo systemctl status apache2
```

Instala-se, depois, o MySQL, e corre-se o assistente de configuração de segurança, que pede para definir uma password de administrador e remover configurações inseguras por omissão:

```
sudo apt install mysql-server -y  
sudo mysql_secure_installation
```

Por fim, instala-se o PHP, juntamente com o módulo que liga o PHP ao Apache, e a extensão que permite ao PHP comunicar com o MySQL:

```
sudo apt install php libapache2-mod-php php-mysql -y
sudo systemctl restart apache2
```

O reinício do Apache, no final, é necessário para que este passe a reconhecer o módulo do PHP instalado, e a interpretar corretamente ficheiros `.php`.

A.4.1.3. Testar a Instalação

O Apache, por omissão, serve os ficheiros guardados na pasta `/var/www/html`. Para confirmar que o PHP está corretamente instalado, cria-se ali um pequeno ficheiro de teste:

```
echo "<?php phpinfo(); ?>" | sudo tee /var/www/html/info.php
```

Ao abrir `http://localhost/info.php` no *browser*, deve aparecer uma página com informação detalhada sobre a instalação do PHP, incluindo a sua versão e a lista de extensões ativas, onde deve constar `mysqli`. Depois de confirmado, este ficheiro deve ser apagado, já que expõe informação detalhada do servidor que não deve ficar acessível fora de um ambiente de desenvolvimento:

```
sudo rm /var/www/html/info.php
```

A.4.1.4. Instalar o phpMyAdmin (Opcional)

O phpMyAdmin, referido no Secção A.1.2, é uma ferramenta de administração do MySQL através do *browser*, útil sobretudo enquanto se aprende, para ver rapidamente o conteúdo de uma tabela sem escrever uma única instrução SQL. Instala-se com:

```
sudo apt install phpmyadmin -y
```

Durante a instalação, o `apt` pergunta qual o servidor web a configurar automaticamente: selecciona-se `apache2` com a barra de espaços, e confirma-se com `Tab` e `Enter`. Pergunta ainda se deve configurar a base de dados do próprio phpMyAdmin com `dbconfig-common`, ao que se responde afirmativamente, definindo uma password para essa configuração interna. Concluída a instalação, o phpMyAdmin fica acessível em `http://localhost/phpmyadmin`, onde se entra com o utilizador e a password já criados no MySQL. Se o Apache não for configurado automaticamente, pode ser necessário acrescentar manualmente a linha `Include /etc/phpmyadmin/apache.conf` à sua configuração, e reiniciá-lo:

```
echo "Include /etc/phpmyadmin/apache.conf" | sudo tee -a /etc/apache2/apache2.conf
sudo systemctl restart apache2
```

A.4.1.5. Editar Código a Partir do Computador Anfitrião

Trabalhar diretamente dentro da máquina virtual, num editor de texto instalado ali, é possível, mas pouco confortável. Duas alternativas permitem editar o código com o Visual Studio Code (code.visualstudio.com/download) instalado no computador anfitrião, mantendo os ficheiros acessíveis ao Apache dentro da máquina virtual:

- **Pasta partilhada:** nas definições da máquina virtual, no VirtualBox, é possível definir uma pasta do computador anfitrião como pasta partilhada, montada dentro do Ubuntu. Editando os ficheiros nessa pasta a partir do anfitrião, as alterações ficam imediatamente visíveis dentro da máquina virtual;
- **Ligação remota por SSH:** instalando um servidor SSH dentro da máquina virtual (`sudo apt install openssh-server`), a extensão **Remote - SSH** do Visual Studio Code, no computador anfitrião, permite abrir e editar diretamente os ficheiros da máquina virtual, como se estivessem no próprio computador.

A.4.2. Opção 2: Contentores com Docker

A.4.2.1. Definir os Serviços com Docker Compose

Com o Docker instalado, tal como descrito no Secção A.3.1, em vez de instalar o Apache, o PHP e o MySQL manualmente, como na opção anterior, descreve-se, num ficheiro `docker-compose.yml`, os contentores necessários e como comunicam entre si, a partir da imagem oficial do PHP (hub.docker.com/_/php) e da imagem oficial do MySQL (hub.docker.com/_/mysql). A imagem oficial do PHP com Apache não inclui, por omissão, a extensão de ligação ao MySQL, pelo que é necessário um pequeno Dockerfile próprio para a acrescentar:

```
# Dockerfile
FROM php:8.3-apache
RUN docker-php-ext-install mysqli pdo_mysql
```

```
# docker-compose.yml
services:
  web:
    build: .
    ports:
      - "8080:80"
    volumes:
      - ./src:/var/www/html

  db:
    image: mysql:8
    environment:
      MYSQL_ROOT_PASSWORD: root
      MYSQL_DATABASE: aplicacao
    ports:
      - "3306:3306"
    volumes:
      - db_data:/var/lib/mysql
```

```
volumes:  
  db_data:
```

Note-se a linha `volumes: - ./src:/var/www/html` do serviço `web`: liga diretamente a pasta `src`, no computador anfitrião, à pasta onde o `Apache` serve ficheiros dentro do contentor, o que significa que qualquer ficheiro `.php` criado ou editado nessa pasta, com o `Visual Studio Code` a correr normalmente no anfitrião, fica imediatamente disponível ao servidor, sem qualquer partilha ou ligação remota adicional.

Com o `Dockerfile` e o `docker-compose.yml` na mesma pasta, os contentores constroem-se e arrancam-se com:

```
docker compose up -d --build
```

O `Apache`, dentro do contentor `web`, fica acessível no computador anfitrião em `http://localhost:8080`, e o `MySQL`, dentro do contentor `db`, fica acessível em `localhost`, na porta 3306, com o utilizador `root` e a password definida na variável `MYSQL_ROOT_PASSWORD`.

A.4.2.2. Adicionar o phpMyAdmin (Opcional)

O `phpMyAdmin`, referido no Secção A.1.2, também está disponível como imagem oficial no `Docker Hub` (hub.docker.com/_/phpmyadmin), bastando acrescentar mais um serviço ao mesmo `docker-compose.yml`, ligado ao contentor `db` através da variável `PMA_HOST`:

```
# docker-compose.yml (serviço adicional)  
phpmyadmin:  
  image: phpmyadmin:latest  
  environment:  
    PMA_HOST: db  
  ports:  
    - "8081:80"  
  depends_on:  
    - db
```

Depois de reiniciar os contentores com `docker compose up -d`, o `phpMyAdmin` fica acessível em `http://localhost:8081`, onde se entra com o utilizador `root` e a password definida em `MYSQL_ROOT_PASSWORD`. Ao contrário da instalação na máquina virtual, não há aqui qualquer configuração adicional do `Apache` a fazer, já que o `phpMyAdmin` corre no seu próprio contentor, independente do contentor `web`.

A.4.2.3. Editar Código a Partir do Computador Anfitrião

Ao contrário da opção da máquina virtual, esta configuração já liga diretamente a pasta `src` do anfitrião ao contentor, pelo que basta abrir essa pasta normalmente no `Visual Studio Code`, sem necessidade de pastas partilhadas ou ligação remota por `SSH`. Opcionalmente, a extensão `Dev Containers` do `Visual Studio Code` permite ainda abrir o próprio editor “dentro” do contentor `web`, com acesso direto às ferramentas ali instaladas.

A.5. Verificar a Instalação

Seja qual for a opção escolhida, e seja o ambiente fornecido pelo professor ou construído do zero, o teste final é o mesmo: confirmar que o PHP consegue efetivamente comunicar com o MySQL. Cria-se, na pasta servida pelo Apache (`/var/www/html` na máquina virtual, ou `src` nos contentores), um pequeno ficheiro de teste:

```
<?php
$conexao = mysqli_connect("localhost", "root", "root");

if ($conexao) {
    echo "Ligação ao MySQL estabelecida com sucesso!";
} else {
    echo "Erro ao ligar: " . mysqli_connect_error();
}
```

Ao abrir este ficheiro no *browser*, deve aparecer a mensagem de sucesso. Se aparecer, em alternativa, uma mensagem de erro, os motivos mais comuns são o MySQL ainda não ter arrancado por completo, a password indicada não corresponder à definida durante a instalação, ou, no caso dos contentores, o nome do servidor não ser `localhost` mas sim o nome do serviço definido no `docker-compose.yml` (`db`, no exemplo acima), já que, de dentro de um contentor, `localhost` refere-se ao próprio contentor, e não a outros contentores da mesma aplicação.

Se o `phpMyAdmin` estiver disponível, referido no Secção A.1.2, este mesmo teste pode ser feito de forma visual, sem escrever nenhum código: basta autenticar-se com as credenciais do MySQL e confirmar que a base de dados criada aparece listada.

Com este teste bem-sucedido, o ambiente está pronto para o desenvolvimento de código PHP retomado no Secção 3.2.

Referências

- Bytebase. 2026. «Drizzle ORM vs Prisma: Which TypeScript ORM Should You Use in 2026?» 2026. <https://www.bytebase.com/blog/drizzle-vs-prisma/>.
- DreamFactory. 2025. «45 SOAP to REST Conversion Statistics Every Enterprise Should Know». 2025. <https://www.dreamfactory.com/hub/soap-to-rest-conversion-statistics>.
- Google. s.d. «Cloud Firestore Data Model». <https://firebase.google.com/docs/firestore/data-model>.
- JetBrains. 2025. «The State of PHP 2025». 2025. <https://blog.jetbrains.com/phpstorm/2025/10/state-of-php-2025/>.
- Mapsoft. s.d. «Adobe XD End of Life: Status, Timeline & Alternatives». <https://mapsoft.com/adobe-xd-end-of-life/>.
- «Native CSS Nesting: Sass Syntax With No Build Step (And One Big Trap)». 2025. 2025. <https://www.w3tweaks.com/css/css-nesting-explained/>.
- OWASP Foundation. 2026. «OWASP Top 10:2025». 2026. <https://owasp.org/Top10/2025/>.
- Patchstack. 2025. «Patchstack's 2025 WordPress Security Report: Mid-Year Vulnerability Breakdown». 2025. <https://patchstack.com/whitepaper/2025-mid-year-vulnerability-report/>.
- Reenskaug, Trygve. 2003. «MVC: Xerox PARC 1978-79». 2003. <https://citeseerx.ist.psu.edu/document?repid=rep1&type=pdf&doi=4ef90a7b9c1b1cd02acf273694e4059a70c7d198>.

Referências

- «State of JavaScript 2025: Front-end Frameworks». 2025. 2025. <https://2025.stateofjs.com/en-US/libraries/front-end-frameworks/>.
- «Top CSS Frameworks in 2025: What Devs Actually Use». 2025. 2025. <https://tailkits.com/blog/popular-css-frameworks/>.

B. Demos de Código das Aulas

B.1. Introdução

Este apêndice reúne o código completo das demonstrações feitas ao vivo nas aulas teóricas, uma por cada grande tema do PHP abordado nos capítulos anteriores. Cada demo aqui apresentada foi limpa da parte puramente decorativa (o HTML e a CSS que envolvem a demonstração na aula), mantendo apenas a lógica em PHP e os comentários necessários para a compreender, de modo a que possa ser copiada diretamente para um ficheiro `.php` próprio e executada no ambiente preparado no Secção A.1, sem qualquer alteração. Cada capítulo do texto principal remete para a secção correspondente deste apêndice, no ponto em que essa demo é relevante.

B.2. Demo: Variáveis e Arrays

Demonstração usada no Secção 3.4 e no Secção 3.6, com exemplos dos quatro tipos de arrays vistos nesse capítulo: numérico, associativo e multidimensional.

```
<?php
// Declarar variáveis de tipos diferentes
$idade = 19; // Inteiro
$preco_do_almoco = 10.5; // Float
$texto = "Olá, Mundo!"; // String
$porta_aberta = true; // Booleano
$arraySimples = array(1, 2, 3); // Array

// Mostrar os valores de forma legível
echo "<p><strong>Idade:</strong> $idade</p>";
echo "<p><strong>Preço do almoço:</strong> $preco_do_almoco</p>";
echo "<p><strong>Texto:</strong> $texto</p>";

// echo de um booleano mostra "1" para true e nada para false;
// para mostrar um texto legível, usa-se uma expressão ternária
echo "<p><strong>Porta aberta:</strong> " . ($porta_aberta ? 'true' : 'false') . "</p>";
```

```
// echo direto de um array produz um Warning; usa-se print_r() ou var_dump()
echo "<p><strong>Array simples:</strong> ";
print_r($arraySimples);
echo "</p>";

// Array indexado
$frutas = array("Maçã", "Banana", "Laranja");
echo "<p>Primeira fruta: " . $frutas[0] . "</p>";
echo "<p>Segunda fruta: " . $frutas[1] . "</p>";
echo "<p>Terceira fruta: " . $frutas[2] . "</p>";

// Array associativo
$pessoa = array(
    "nome"    => "João",
    "idade"   => 25,
    "cidade"  => "Lisboa"
);
echo "<p>Nome: " . $pessoa["nome"] . "</p>";
echo "<p>Idade: " . $pessoa["idade"] . "</p>";
echo "<p>Cidade: " . $pessoa["cidade"] . "</p>";

// Array multidimensional
$carros = array(
    "desportivos" => array("Ferrari", "Lamborghini", "Porsche"),
    "suv"          => array("BMW X5", "Audi Q7", "Mercedes GLE"),
    "sedans"       => array("Toyota Camry", "Honda Accord", "Hyundai Elantra")
);
echo "<p>Primeiro carro desportivo: " . $carros["desportivos"][0] . "</p>";
echo "<p>Segundo SUV: " . $carros["suv"][1] . "</p>";
echo "<p>Terceiro sedan: " . $carros["sedans"][2] . "</p>";
```

B.3. Demo: Condicionais, Loops e Supervariáveis

Demonstração usada no Secção 4.1, no Secção 4.2 e no Secção 4.3. A parte de \$_POST requer um pequeno formulário HTML na mesma página, submetido para si próprio (action="").

```
<?php
// --- Condicionais ---
$idade = 18;

if ($idade >= 18) {
    echo "<p>É maior de idade.</p>";
} else {
    echo "<p>É menor de idade.</p>";
}

$cor = "vermelho";

switch ($cor) {
```

```
case "azul":
    echo "<p>A cor é azul.</p>";
    break;
case "vermelho":
    echo "<p>A cor é vermelho.</p>";
    break;
default:
    echo "<p>A cor não é azul nem vermelho.</p>";
}

// --- Loops ---
echo "<h3>Loop for</h3>";
for ($i = 1; $i <= 5; $i++) {
    echo "<p>Iteração $i</p>";
}

echo "<h3>Loop while</h3>";
$contador = 1;
while ($contador <= 3) {
    echo "<p>Contagem: $contador</p>";
    $contador++;
}

echo "<h3>Loop foreach</h3>";
$frutas = array("Maçã", "Banana", "Laranja");
foreach ($frutas as $fruta) {
    echo "<p>Fruta: $fruta</p>";
}
?>

<!-- --- Supervariáveis: $_GET, $_POST, $_SERVER --- -->
<form method="post" action="">
    <label for="nome">Nome:</label>
    <input type="text" id="nome" name="nome">
    <input type="submit" value="Enviar">
</form>

<?php
// $_GET: dados vindos de um parâmetro na URL (ex: ?nome=Ana)
echo "<p>Valor de 'nome' via GET: "
    . (isset($_GET['nome']) ? $_GET['nome'] : 'não fornecido') . "</p>";

// $_POST: dados vindos do formulário acima, submetido por POST
if ($_SERVER["REQUEST_METHOD"] === "POST") {
    echo "<p>Nome enviado via POST: " . htmlspecialchars($_POST['nome']) . "</p>";
}

// $_SERVER: informação sobre o próprio servidor e o pedido
echo "<p>Nome do servidor: " . $_SERVER['SERVER_NAME'] . "</p>";
```

B.4. Demo: Funções

Demonstração usada no Secção 5.1, com quatro variantes de função (sem parâmetros nem retorno, com parâmetros, com retorno, e com parâmetro opcional).

```
<?php
// Sem parâmetros e sem retorno
function exibirSaudacao() {
    echo "<p>Bem-vindo ao sistema!</p>";
}

// Com parâmetros e sem retorno
function enviarEmail($destinatario, $assunto, $mensagem) {
    echo "<p>Email enviado para: $destinatario</p>";
    echo "<p>Assunto: $assunto</p>";
    echo "<p>Mensagem: $mensagem</p>";
}

// Com parâmetros e com retorno
function calcularDesconto($preco, $descontoPercentual) {
    $desconto = $preco * ($descontoPercentual / 100);
    return $preco - $desconto;
}

// Com parâmetro opcional (valor por omissão)
function criarUtilizador($nome, $email, $senha, $role = 'user') {
    echo "<p>Utilizador $nome criado com a função de $role.</p>";
}

function volume($largura, $altura, $comprimento) {
    return $largura * $altura * $comprimento;
}

// --- Chamadas ---
$volume_da_sala = volume(5, 3, 10);
echo "<p>O volume da sala é de $volume_da_sala m3.</p>";

exibirSaudacao();

enviarEmail("cliente@example.com", "Bem-vindo", "Obrigado por se registar!");

$precoComDesconto = calcularDesconto(100, 10); // 10% de desconto
echo "<p>Preço final com desconto: $precoComDesconto</p>";

criarUtilizador("José", "jose@example.com", "123456"); // sem 'role': usa 'user'
criarUtilizador("Ana", "ana@example.com", "789123", "admin"); // 'role' explícito
```

B.5. Demo: Datas

Demonstração usada no Secção 5.2, desta vez com a classe `DateTime`, orientada a objetos, em vez da função `date()`.

```
<?php
// Criar e mostrar a data e hora atuais
$dataAtual = new DateTime();
echo "<p>Data e hora atuais: " . $dataAtual->format('d-m-Y H:i:s') . "</p>";

// Somar dias a uma data (clonar para não alterar o objeto original)
$dataFutura = clone $dataAtual;
$dataFutura->modify('+150 days');
echo "<p>Data daqui a 150 dias: " . $dataFutura->format('Y-m-d') . "</p>";

// Subtrair dias a uma data
$dataPassada = clone $dataAtual;
$dataPassada->modify('-10 days');
echo "<p>Data há 10 dias: " . $dataPassada->format('Y-m-d') . "</p>";

// Diferença entre duas datas
$intervalo = $dataAtual->diff($dataPassada);
echo "<p>Diferença: " . $intervalo->days . " dias</p>";

// Criar uma data a partir de uma string, num formato específico
$dataEspecificas = DateTime::createFromFormat('Y-m-d', '2024-12-25');
echo "<p>Data específica (Natal): " . $dataEspecificas->format('Y-m-d') . "</p>";

// Verificar se um ano é bissexto
$ano = (int) $dataEspecificas->format('Y');
if ($ano % 4 === 0 && ($ano % 100 !== 0 || $ano % 400 === 0)) {
    echo "<p>$ano é um ano bissexto.</p>";
} else {
    echo "<p>$ano não é um ano bissexto.</p>";
}
```

B.6. Demo: Includes (Header e Footer)

Demonstração usada no Secção 5.3, com três ficheiros: um `header.php` e um `footer.php` reutilizáveis, incluídos por uma página principal.

```
<!-- header.php -->
<header>
    <h1>Bem-vindo à minha aplicação!</h1>
    <nav>
        <a href="index.php">Home</a> |
        <a href="sobre.php">Sobre</a> |
        <a href="contacto.php">Contacto</a>
    </nav>
</header>
```

```
<!-- footer.php -->
<footer>
    <p>&copy; 2026 A Minha Aplicação. Todos os direitos reservados.</p>
</footer>
```

```
<!-- index.php -->
<?php
// include: se header.php não existir, é apenas um Warning; a página continua
include 'header.php';
?>

<h2>Página Principal</h2>
<p>Conteúdo específico desta página.</p>

<?php
// require: se footer.php não existir, é um erro fatal; a página para aqui
require 'footer.php';
?>
```

Referências

- Bytebase. 2026. «Drizzle ORM vs Prisma: Which TypeScript ORM Should You Use in 2026?» 2026. <https://www.bytebase.com/blog/drizzle-vs-prisma/>.
- DreamFactory. 2025. «45 SOAP to REST Conversion Statistics Every Enterprise Should Know». 2025. <https://www.dreamfactory.com/hub/soap-to-rest-conversion-statistics>.
- Google. s.d. «Cloud Firestore Data Model». <https://firebase.google.com/docs/firestore/data-model>.
- JetBrains. 2025. «The State of PHP 2025». 2025. <https://blog.jetbrains.com/phpstorm/2025/10/state-of-php-2025/>.
- Mapsoft. s.d. «Adobe XD End of Life: Status, Timeline & Alternatives». <https://mapsoft.com/adobe-xd-end-of-life/>.
- «Native CSS Nesting: Sass Syntax With No Build Step (And One Big Trap)». 2025. 2025. <https://www.w3tweaks.com/css/css-nesting-explained/>.
- OWASP Foundation. 2026. «OWASP Top 10:2025». 2026. <https://owasp.org/Top10/2025/>.
- Patchstack. 2025. «Patchstack’s 2025 WordPress Security Report: Mid-Year Vulnerability Breakdown». 2025. <https://patchstack.com/whitepaper/2025-mid-year-vulnerability-report/>.
- Reenskaug, Trygve. 2003. «MVC: Xerox PARC 1978-79». 2003. <https://citeseerx.ist.psu.edu/document?repid=rep1&type=pdf&doi=4ef90a7b9c1b1cd02acf273694e4059a70c7d198>.
- «State of JavaScript 2025: Front-end Frameworks». 2025. 2025. <https://2025.stateofjs.com/en-US/libraries/front-end-frameworks/>.
- «Top CSS Frameworks in 2025: What Devs Actually Use». 2025. 2025. <https://tailkits.com/blog/popular-css-frameworks/>.